

VIETNAM NATIONAL UNIVERSITY HO CHI MINH
CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND
ENGINEERING



GRADUATION THESIS

THESIS

**BUILDING A FRAMEWORK TO
GENERATE AI MODELS FOR
HUMAN MOTION TRACKING
APPLICATIONS**

Major: Computer Science

SUPERVISOR(s): Dr. Le Trong Nhan

—o0o—

STUDENT: Nguyen Truong Minh Hoang

STUDENT'S ID: 2270757

HO CHI MINH CITY, JANUARY 2026

VERIFICATION OF INSTRUCTOR

Signature confirmation

LỜI CAM ĐOAN

Em cam đoan rằng nghiên cứu này là công trình của riêng em, được thực hiện dưới sự giám sát và hướng dẫn của TS. Lê Trọng Nhân. Kết quả nghiên cứu của em là hợp pháp và chưa được công bố dưới bất kỳ hình thức nào trước đây. Tất cả các tài liệu được sử dụng trong nghiên cứu này đều được em thu thập từ nhiều nguồn khác nhau và đã được liệt kê đầy đủ trong phần tài liệu tham khảo. Ngoài ra, trong nghiên cứu này, em cũng đã sử dụng kết quả của một số tác giả và tổ chức khác. Tất cả đều đã được trích dẫn một cách thích hợp. Trong mọi trường hợp đạo văn, em hoàn toàn chịu trách nhiệm về hành động của mình. Do đó, Trường Đại học Bách Khoa - ĐHQG TP.HCM không chịu trách nhiệm về bất kỳ vi phạm bản quyền nào được thực hiện trong nghiên cứu của em.

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn chân thành đến tất cả những người đã đóng góp vào việc phát triển và hoàn thiện luận văn này.

Trước tiên, em xin gửi lời tri ân sâu sắc nhất đến người hướng dẫn khoa học của em, TS. Lê Trọng Nhân, vì sự hướng dẫn quý báu, động viên và phản hồi mang tính xây dựng trong suốt quá trình hình thành đề tài này. Chuyên môn và những hiểu biết sâu sắc của thầy đã đóng vai trò quan trọng trong việc định hướng nghiên cứu của em.

Em cũng vô cùng biết ơn Trường Đại học Bách Khoa - ĐHQG TP.HCM và Khoa Khoa học và Kỹ thuật Máy tính đã cung cấp môi trường học thuật nuôi dưỡng và các nguồn lực cần thiết để thực hiện nghiên cứu này.

Lời cảm ơn đặc biệt gửi đến các bạn đồng nghiệp và đồng môn đã chia sẻ quan điểm và đưa ra những gợi ý sâu sắc, làm phong phú thêm phạm vi của nghiên cứu này.

Cuối cùng, em vô cùng biết ơn gia đình và bạn bè vì sự ủng hộ không ngừng, sự kiên nhẫn và động lực khi em đắm mình vào nỗ lực đầy thách thức nhưng bổ ích này.

Đề xuất nghiên cứu này là sự phản ánh của sự hỗ trợ và cống hiến tập thể, và em xin chân thành cảm ơn tất cả những người đã đóng góp trực tiếp hoặc gián tiếp vào việc hiện thực hóa nó.

TÓM TẮT

Sự phát triển của học máy và điện toán biên đã cách mạng hóa các ứng dụng theo dõi chuyển động, cho phép các hệ thống thời gian thực, tiết kiệm năng lượng và chính xác cao cho nhiều trường hợp sử dụng trong y tế, phân tích thể thao và tương tác người-máy. Tuy nhiên, việc phát triển các mô hình AI cho theo dõi chuyển động vẫn còn phức tạp, đòi hỏi kiến thức chuyên môn về tiền xử lý dữ liệu, trích xuất đặc trưng, huấn luyện mô hình và triển khai trên thiết bị biên. Bài báo này trình bày một framework toàn diện giúp đơn giản hóa quy trình end-to-end để tạo ra các mô hình AI được tùy chỉnh cho theo dõi chuyển động con người trên các thiết bị biên có tài nguyên hạn chế.

Framework giới thiệu một phương pháp tiền xử lý tương tác mới với tính năng lựa chọn cửa sổ thời gian kéo thả, cho phép người dùng chú thích và phân đoạn dữ liệu chuyển động một cách hiệu quả mà không cần kiến thức lập trình sâu rộng. Hệ thống hỗ trợ nhiều thuật toán học máy bao gồm Random Forest, Support Vector Machines và Neural Networks, với khả năng tự động sinh mã cho các nền tảng biên khác nhau bao gồm vi điều khiển Arduino, ARM Cortex-M và Seeed XIAO. Việc sinh mã nhận biết tối ưu hóa của chúng tôi xem xét nhiều chiến lược triển khai—độ chính xác, tốc độ, tiêu thụ điện năng và các phương pháp cân bằng—điều chỉnh mã được sinh ra theo yêu cầu ứng dụng cụ thể.

Chúng tôi xác thực framework bằng cách sử dụng bộ dữ liệu Nhận dạng Hoạt động Con người (HAR) được thu thập ở 100Hz từ cảm biến IMU 6 trục (LSM6DS3) trên nền tảng Seeed XIAO nRF52840 Sense. Trong nghiên cứu bằng chứng khái niệm này sử dụng dữ liệu đơn đối tượng trên năm lớp hoạt động, kết quả thực nghiệm cho thấy các mô hình đã triển khai đạt độ chính xác 94.5% cho Random Forest và 96.2% cho Neural Networks, với thời gian suy luận lần lượt là 15ms và 12ms, trong khi tiêu thụ ít hơn 10mW trong quá trình hoạt động. Phân tích so sánh với các nền tảng thương mại (Edge Impulse, SensiML) cho thấy framework mã nguồn mở của chúng tôi cung cấp tính linh hoạt tiền xử lý vượt trội và chất lượng sinh mã tốt hơn với chi phí bằng không.

Framework giải quyết các khoảng trống quan trọng trong các giải pháp hiện tại bằng cách cung cấp: (1) quy trình làm việc dựa trên GUI trực quan dễ tiếp cận cho người không chuyên, (2) kiểm soát hoàn toàn đường ống tiền xử lý với phản hồi trực quan, (3) triển khai không phụ thuộc phần cứng với các tối ưu hóa cụ thể cho từng nền tảng, và (4) kiến trúc có thể mở rộng hỗ trợ tích hợp các thuật toán và nền tảng

bổ sung trong tương lai. Nghiên cứu này đóng góp vào việc dân chủ hóa phát triển AI biên cho các ứng dụng theo dõi chuyển động, với tác động tiềm năng trong phục hồi chức năng y tế, giám sát hiệu suất thể thao và công nghệ hỗ trợ.

Mục lục

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN	iii
TÓM TẮT	iv
Mục lục	vi
Danh sách bảng	x
Danh sách hình vẽ	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Theoretical Foundation	2
1.2.1 Signal Processing Principles	2
1.2.2 Feature Extraction Rationale	2
1.2.3 Biomechanical Basis for Activity Recognition	3
1.3 Problem Statement	3
1.4 Comparison with Existing Platforms	4
1.5 Rationale for Key Design Decisions	6
1.5.1 Interactive Preprocessing Approach	6
1.5.2 Machine Learning Algorithm Selection	6
1.6 Research Objectives	7
1.7 Research Contributions	8
1.8 Scope and Limitations	9
1.9 Thesis Organization	9
2 Related Work	10
2.1 Overview	10

2.2	Application Domains of Motion Tracking	10
2.3	Sensor-Based Motion Tracking	11
2.4	Vision-Based Motion Tracking	11
2.5	AI in Motion Tracking	12
2.6	Model Generation Frameworks	12
2.7	Data Preprocessing and Windowing	13
2.8	Feature Extraction and Representation	14
2.9	Edge Deployment and Optimization	15
2.10	Gaps in Existing Work	15
2.11	Summary	16
3	Methodology	17
3.1	Framework Architecture Overview	17
3.2	Data Collection and Hardware Platform	17
3.2.1	Sensor Hardware	17
3.2.2	Data Acquisition Protocol	17
3.3	Interactive Preprocessing Pipeline	18
3.3.1	Data Upload and Visualization	18
3.3.2	Signal Cleaning and Smoothing	20
3.3.3	Draggable Time Window Segmentation	20
3.4	Feature Extraction	21
3.4.1	Sliding Window Approach	21
3.4.2	Feature Set	22
3.5	Machine Learning Model Training	23
3.5.1	Supported Algorithms	23
3.5.2	Training and Evaluation Protocol	24
3.5.3	Hyperparameter Optimization	25
3.6	Code Generation for Edge Deployment	27
3.6.1	Architecture	27
3.6.2	Generated Code Structure	28
3.6.3	Optimization Strategies	29
3.6.4	Platform-Specific Adaptations	30
3.7	Experimental Design	31
3.7.1	Dataset	31
3.7.2	Evaluation Metrics	31
3.7.3	Baseline Comparisons	32

3.8	Implementation Details	33
4	Results	34
4.1	Overview	34
4.2	Model Training Performance	34
4.2.1	Overall Accuracy	34
4.2.2	Per-Class Performance	35
4.2.3	Confusion Matrix Analysis	35
4.3	Impact of Class Balancing	38
4.3.1	Ablation Study	38
4.4	Edge Deployment Characteristics	39
4.4.1	Inference Time and Latency	39
4.4.2	Optimization Mode Trade-offs	39
4.4.3	Memory Footprint	40
4.4.4	Power Consumption	41
4.5	Comparative Analysis	42
4.5.1	Comparison with Commercial Platforms	42
4.5.2	Hyperparameter Optimization Impact	43
4.6	Summary of Results	43
5	Discussion	45
5.1	Interpretation of Key Findings	45
5.1.1	Model Performance and Algorithm Selection	45
5.1.2	Class Imbalance: A Critical but Overlooked Challenge	46
5.1.3	Edge Deployment Feasibility	46
5.1.4	Interactive Preprocessing: A Paradigm Shift	47
5.2	Comparison with Related Work	47
5.2.1	Commercial Platforms	47
5.2.2	Academic HAR Systems	48
5.3	Limitations and Threats to Validity	48
5.3.1	Dataset Limitations	48
5.3.2	Window Size Selection	49
5.3.3	Sensor Placement and Orientation	49
5.3.4	Computational Platform Diversity	49
5.3.5	External Validity	49
5.4	Design Trade-offs and Alternatives	50
5.4.1	Classical ML vs Deep Learning	50

5.4.2	Handcrafted vs Learned Features	50
5.4.3	Optimization Modes	50
5.5	Implications for Practice	51
5.5.1	Research and Education	51
5.5.2	Healthcare Applications	51
5.5.3	Sports and Fitness	52
5.6	Future Research Directions	52
5.6.1	Short-Term Extensions	52
5.6.2	Long-Term Vision	52
5.7	Summary	53
6	Conclusion	54
6.1	Summary of Contributions	54
6.1.1	Technical Contributions	54
6.1.2	Empirical Validation	55
6.1.3	Broader Impact	55
6.2	Revisiting Research Objectives	56
6.3	Practical Applications	56
6.4	Limitations and Future Work	57
6.4.1	Dataset Diversity	57
6.4.2	Algorithmic Extensions	58
6.4.3	Platform Expansion	58
6.4.4	Advanced Features	58
6.4.5	Usability Enhancements	59
6.5	Closing Remarks	59
6.6	Final Thoughts	60
	Tài liệu tham khảo	61

Danh sách bảng

1.4.1	Comparison of Edge ML Platforms for Motion Tracking	5
4.2.1	Overall Model Accuracy on Test Set (5-class HAR, Single Subject)	34
4.2.2	Per-Class Performance Metrics (Neural Network Model, 5 Activities) . .	35
4.3.1	Ablation Study: Impact of Class Balancing on Minority Classes	38
4.4.1	Inference Timing on Seeed XIAO nRF52840 (per 75-sample window) . .	39
4.4.2	Memory Usage on Seeed XIAO nRF52840 (1MB Flash, 256KB RAM) . .	40
4.5.1	Framework Comparison: Proposed vs Commercial Solutions (Estimated)	42
4.5.2	Impact of Hyperparameter Optimization (5-fold CV)	43

Danh sách hình vẽ

1.6.1	Complete User Workflow: From data upload through deployment (estimated 30-60 minutes total)	7
3.1.1	Framework Architecture Overview: End-to-end pipeline from data upload to edge deployment	18
3.3.1	Web interface for CSV file upload and dataset management. The interface allows users to upload sensor data files, validates column names, automatically detects sampling frequency, and provides an interactive preview of the raw six-axis time series.	19
3.3.2	Interactive draggable window interface for activity segmentation. Users visually inspect accelerometer and gyroscope time series, drag rectangular windows over regions corresponding to distinct activities, assign activity labels, and adjust boundaries via click-and-drag interaction. This eliminates manual timestamp calculation and makes the framework accessible to non-programmers.	21
3.5.1	Model configuration panel showing algorithm selection (Random Forest, Neural Network, SVM), hyperparameter tuning options with GridSearchCV, class balancing strategies (balanced weights, stratified splitting), and real-time training progress monitoring with cross-validation scores.	27
3.6.1	Generated Arduino C++ code preview interface. The framework displays the generated header files, implementation files, and Arduino sketches with syntax highlighting. Users can select optimization modes (Accuracy, Speed, Power, Balanced), configure target platform parameters, and download the complete deployment package.	29
4.2.1	Confusion Matrix for Neural Network Model (5-class HAR, 96.2% accuracy, single-subject validation)	36

4.2.2	Results dashboard displaying confusion matrix and performance metrics. The interface provides interactive visualizations including confusion matrices with heatmaps, per-class precision/recall/F1-scores, overall accuracy metrics, ROC curves, and downloadable performance reports. Users can compare multiple models side-by-side.	37
4.4.1	Comparative analysis of accuracy, inference latency, and power consumption across four optimization modes	40
4.4.2	Power Consumption Profile During Inference (Neural Network, Accuracy Mode)	41

Chương 1

Giới thiệu

1.1 Bối cảnh và Động lực Nghiên cứu

Theo dõi chuyển động (Motion tracking) là một lĩnh vực năng động và phát triển nhanh chóng, liên quan đến việc thu thập, xử lý và phân tích chuyển động của các đối tượng, cá nhân hoặc môi trường. Công nghệ này đóng vai trò then chốt trong nhiều lĩnh vực bao gồm y tế (phục hồi chức năng, phát hiện té ngã, giám sát bệnh nhân), thể thao (theo dõi hiệu suất, phòng ngừa chấn thương), trò chơi điện tử và thực tế ảo/thực tế tăng cường (trải nghiệm người dùng nhập vai), và robot học (hệ thống định vị, cơ chế điều khiển chính xác)^[37].

Thị trường toàn cầu cho công nghệ theo dõi chuyển động đạt 2,8 tỷ USD vào năm 2023 và dự kiến tăng trưởng với tốc độ tăng trưởng kép hàng năm (CAGR) là 18,4% đến năm 2030^[1]. Sự tăng trưởng này được thúc đẩy đặc biệt bởi các thiết bị đeo y tế và ứng dụng nhà thông minh. Sự quan tâm trong giới học thuật cũng tăng vọt, với các công bố nghiên cứu về nhận dạng hoạt động con người (Human Activity Recognition - HAR) tăng từ 487 bài báo vào năm 2018 lên hơn 1.500 bài vào năm 2023, tương ứng với mức tăng 3,1 lần chỉ trong năm năm^[31]. Sự hội tụ của các cảm biến IoT giá cả phải chăng, thuật toán học máy hiệu quả và khả năng tính toán biên (edge computing) đã dân chủ hóa việc theo dõi chuyển động từ các phòng thí nghiệm chuyên biệt sang các ứng dụng tiêu dùng hàng ngày.

Những tiến bộ trong công nghệ cảm biến và trí tuệ nhân tạo (AI) đã mang tính chuyển đổi, làm tăng đáng kể độ chính xác, hiệu suất và tính linh hoạt của các hệ thống theo dõi chuyển động. Đặc biệt, các mô hình AI là không thể thiếu trong việc diễn giải dữ liệu chuyển động, cho phép các ứng dụng như nhận dạng cử chỉ, phân loại hoạt động và mô hình hóa chuyển động dự đoán. Tuy nhiên, quá trình phát triển các mô hình AI này vốn dĩ phức tạp, bao gồm thu thập dữ liệu, tiền xử lý và lọc để loại bỏ

nhiều, trích xuất các đặc trưng có ý nghĩa, và huấn luyện các mô hình học máy hoặc học sâu. Sự phức tạp này đặt ra những thách thức đáng kể, đặc biệt đối với các nhà phát triển và nhà nghiên cứu có thể thiếu chuyên môn về AI hoặc công nghệ theo dõi chuyển động^[21].

Sự phát triển của tính toán biên (edge computing) đã làm phức tạp thêm bối cảnh này trong khi đồng thời mang lại những cơ hội mới. Các thiết bị biên—vi điều khiển bị giới hạn tài nguyên với bộ nhớ, sức mạnh xử lý và ngân sách năng lượng hạn chế—cho phép các ứng dụng theo dõi chuyển động theo thời gian thực, bảo vệ quyền riêng tư và tiết kiệm năng lượng. Tuy nhiên, triển khai các mô hình AI trên các thiết bị như vậy đòi hỏi kiến thức chuyên biệt về tối ưu hóa mô hình, tạo mã và các chi tiết triển khai đặc thù cho từng nền tảng.

1.2 Nền tảng Lý thuyết

1.2.1 Nguyên lý Xử lý Tín hiệu

Hiệu quả của việc theo dõi chuyển động dựa trên IMU được căn cứ trên lý thuyết xử lý tín hiệu cơ bản. Chuyển động của con người thể hiện nội dung tần số đặc trưng có thể được thu thập và phân tích một cách đáng tin cậy. Các nghiên cứu về sinh cơ học con người cho thấy hầu hết các hoạt động hàng ngày tạo ra tần số chuyển động dưới 20 Hz, với hoạt động đi bộ thường xảy ra ở 1-2 Hz (tần số bước chân) và chạy ở 2-3 Hz^[33]. Định lý lấy mẫu Nyquist-Shannon nêu rõ rằng để tái tạo chính xác một tín hiệu, tần số lấy mẫu phải ít nhất gấp đôi thành phần tần số cao nhất^[20]. Do đó, tốc độ lấy mẫu 100 Hz cung cấp bằng chứng đủ (tần số Nyquist 50 Hz) để bắt giữ chuyển động của con người với biên độ đáng kể, ngăn chặn hiện tượng aliasing trong khi vẫn có thể tính toán được cho các hệ thống nhúng.

Gia tốc kế đo gia tốc tuyến tính theo ba trục trực giao, thu thập cả gia tốc trọng trường ($9,81 \text{ m/s}^2$) và các thành phần chuyển động động. Con quay hồi chuyển đo vận tốc góc (tốc độ quay), cung cấp thông tin bổ sung về các thay đổi hướng của cơ thể. Sự kết hợp của dữ liệu IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục) tạo ra một biểu diễn phong phú về chuyển động của con người mà các thuật toán học máy có thể khai thác để phân biệt các hoạt động^[12].

1.2.2 Lý do Trích xuất Đặc trưng

Dữ liệu cảm biến thô, mặc dù giàu thông tin, chứa sự dư thừa và nhiều cản trở hiệu suất phân loại. Trích xuất đặc trưng chuyển đổi dữ liệu chuỗi thời gian nhiều

chiều thành các biểu diễn thống kê gọn nhẹ nhấn mạnh các đặc tính phân biệt. Các đặc trưng miền thời gian (trung bình, phương sai, độ lệch, độ nhọn) bắt giữ các thuộc tính phân phối, trong khi các đặc trưng miền tần số có nguồn gốc từ biến đổi Fourier tiết lộ các mẫu tuần hoàn^[20]. Việc giảm chiều này là cần thiết cho triển khai nhúng, vì việc tính toán dự đoán trên 90 đặc trưng được chọn cẩn thận đòi hỏi ít bộ nhớ và tính toán hơn nhiều bậc so với việc xử lý trực tiếp 450 mẫu thô ($75 \text{ mẫu} \times 6 \text{ trục}$).

Lý thuyết thông tin cung cấp lý do bổ sung: trích xuất đặc trưng có thể được xem như nén có mất mát bảo tồn thông tin liên quan đến nhiệm vụ trong khi loại bỏ nhiễu^[9]. Bằng cách chọn các đặc trưng tối đa hóa phương sai giữa các lớp và tối thiểu hóa phương sai trong lớp, chúng ta tạo ra các biểu diễn tạo điều kiện cho khả năng phân tách tuyến tính cho các bộ phân loại.

1.2.3 Cơ sở Sinh cơ học cho Nhận dạng Hoạt động

Các hoạt động khác nhau của con người tạo ra các dấu hiệu sinh cơ học riêng biệt. Đi bộ thể hiện các mẫu gia tốc tuần hoàn với đường cong lực hai đỉnh đặc trưng trong các giai đoạn chạm gót và đẩy ngón chân^[33]. Chạy cho thấy lực tác động cao hơn và các giai đoạn bay dài hơn. Leo cầu thang tạo ra các mẫu gia tốc thẳng đứng không đối xứng. Những khác biệt sinh cơ học này biểu hiện dưới dạng các biến đổi có thể đo được trong dữ liệu cảm biến IMU, cung cấp cơ sở vật lý cho phân loại học máy. Do đó, sự thành công của các hệ thống HAR không dựa vào việc khớp mẫu tùy ý, mà dựa vào việc học các phân biệt dựa trên vật lý về cách cơ thể con người di chuyển trong các hoạt động khác nhau.

1.3 Phát biểu Bài toán

Bất chấp những tiến bộ trong công nghệ theo dõi chuyển động, việc tạo ra các mô hình AI cho phân tích chuyển động vẫn là một nhiệm vụ đầy thách thức đặt ra một số rào cản quan trọng:

1. **Độ phức tạp Kỹ thuật:** Các cách tiếp cận hiện tại đòi hỏi kiến thức kỹ thuật sâu rộng trải rộng nhiều lĩnh vực—xử lý tín hiệu cho tiền xử lý dữ liệu, học máy cho phát triển mô hình và hệ thống nhúng cho triển khai biên. Một khảo sát năm 2023 với 847 nhà phát triển hệ thống nhúng cho thấy 68% đã từ bỏ ít nhất một dự án edge ML do độ phức tạp, với tiền xử lý và tối ưu hóa mô hình được trích dẫn là các rào cản hàng đầu^[18]. Thời gian trung bình từ ý tưởng đến triển khai cho các giải pháp theo dõi chuyển động tùy chỉnh dao động từ 3-6 tháng đối với

các nhóm có kinh nghiệm, với 40% thời gian phát triển chỉ dành cho tiền xử lý dữ liệu^[32].

2. **Thách thức Tiền xử lý:** Dữ liệu cảm biến chuyển động vốn dĩ nhiễu và đòi hỏi tiền xử lý cẩn thận. Các công cụ hiện có hoặc cung cấp kiểm soát hạn chế đối với tiền xử lý (các cách tiếp cận hộp đen tự động) hoặc đòi hỏi lập trình mở rộng để triển khai các quy trình tiền xử lý tùy chỉnh.
3. **Tính linh hoạt Hạn chế:** Nhiều framework có sẵn có đường cong học tập dốc hoặc cung cấp chức năng hạn chế để tùy chỉnh mô hình theo các yêu cầu theo dõi chuyển động cụ thể. Các nền tảng thương mại như Edge Impulse và SensiML cung cấp quy trình làm việc đơn giản hóa nhưng với chi phí đáng kể (\$20-100/tháng) và với các tùy chọn tùy chỉnh bị hạn chế.
4. **Độ phức tạp Triển khai:** Dịch các mô hình đã huấn luyện thành mã tối ưu hóa cho các nền tảng biên cụ thể đòi hỏi hiểu biết sâu về kiến trúc phần cứng đích, ràng buộc bộ nhớ và kỹ thuật tối ưu hóa.
5. **Rào cản Khả năng tiếp cận:** Sự kết hợp của các yếu tố này hạn chế đổi mới và cản trở việc áp dụng rộng rãi các giải pháp theo dõi chuyển động được hỗ trợ bởi AI, đặc biệt trong bối cảnh nghiên cứu và giáo dục.

Do đó, có một nhu cầu cấp thiết về một framework linh hoạt, trực quan và thân thiện với người dùng giúp đơn giản hóa việc tạo các mô hình AI phù hợp cho các ứng dụng theo dõi chuyển động trong khi cung cấp kiểm soát hoàn toàn đối với quy trình phát triển.

1.4 So sánh với các Nền tảng Hiện có

Để đặt đóng góp của chúng tôi trong ngữ cảnh, Bảng 1.4.1 so sánh framework của chúng tôi với các lựa chọn thay thế thương mại và mã nguồn mở hàng đầu cho triển khai edge ML trong các ứng dụng theo dõi chuyển động.

1.4. So sánh với các Nền tảng Hiện có

Bảng 1.4.1: So sánh các Nền tảng Edge ML cho Theo dõi Chuyển động

Tính năng	Edge pulse	Im-	SensiML	TensorFlow Lite	Framework của chúng tôi
Chi phí	\$20-99/tháng		\$49-199/tháng	Miễn phí	Miễn phí (Mã nguồn mở)
Tiền xử lý Tương tác	Hạn chế (web UI)		Có (analytics studio)	Không (chỉ mã)	Có (cửa sổ kéo thả)
Thuật toán	NN, transfer learning		RF, SVM, NN, boosting	Chỉ deep learning	RF, SVM, NN
Tạo Mã	C++ (Arduino, ARM)	(Ar-	C (MCU-cụ thể)	C++ (TFLite Micro)	C++ (đa nền tảng)
Chế độ Tối ưu hóa	Accuracy/latency		AutoML	Quant. thủ công	4 chế độ (acc./speed/power/bal.)
Mất cân bằng Lớp	Oversample thủ công		Cân bằng tự động	Không tích hợp	Tự động (class_weight)
Tùy chỉnh	Hạn chế		Trung bình	Cao (OSS)	Cao (modular)
Sử dụng Giáo dục	Gói miễn phí		Giấy phép học thuật	Có	Có (không giới hạn)
Nền tảng Triển khai	20+ boards		MCUs đã chọn	Tương thích TFLite	Arduino, ARM Cortex-M, mở rộng
Kiểm soát Dữ liệu Huấn luyện	Lưu trữ đám mây		Local/cloud	Chỉ local	Local (bảo mật hoàn toàn)

Các yếu tố phân biệt chính của framework chúng tôi bao gồm: (1) **Tiền xử lý tương tác mới** với chọn cửa sổ thời gian kéo thả, giảm thời gian chú thích so với các cách tiếp cận dựa trên mã hoặc nhấp chuột; (2) **Hỗ trợ đa thuật toán** với các API nhất quán, cho phép thử nghiệm nhanh chóng mà không cần chuyển đổi nền tảng; (3) **Tính minh bạch Hoàn toàn** thông qua tính khả dụng mã nguồn mở, cho phép các nhà nghiên cứu hiểu, xác thực và mở rộng toàn bộ quy trình; (4) **Chi phí bằng không** cho sử dụng không giới hạn, loại bỏ rào cản tài chính cho sinh viên, nhà nghiên

cứu và các khu vực đang phát triển; (5) **Tạo mã nhận thức tối ưu hóa** đánh đổi rõ ràng giữa độ chính xác, tốc độ và tiêu thụ điện năng dựa trên yêu cầu ứng dụng.

Trong khi các nền tảng thương mại cung cấp lợi thế về độ trưởng thành của hệ sinh thái (hỗ trợ 20+ board của Edge Impulse) và sự tinh vi của AutoML (lựa chọn đặc trưng tự động của SensiML), chúng áp đặt chi phí định kỳ (\$240-2.388/năm), hạn chế tùy chỉnh thông qua các API độc quyền và tạo ra sự phụ thuộc nhà cung cấp. TensorFlow Lite cung cấp khả năng học sâu mạnh mẽ nhưng đòi hỏi chuyên môn ML đáng kể và không cung cấp công cụ tiền xử lý cấp cao. Framework của chúng tôi nhắm đến khoảng trống giữa tính dễ sử dụng và tính linh hoạt, cung cấp một giải pháp có thể tiếp cận nhưng mạnh mẽ cho bối cảnh giáo dục và nghiên cứu^[11;13].

1.5 Lý do cho các Quyết định Thiết kế Chính

1.5.1 Phương pháp Tiền xử lý Tương tác

Quyết định triển khai tiền xử lý tương tác dựa trên GUI được căn cứ trên các nguyên tắc tương tác người-máy đã được thiết lập. Phân tích trực quan cho dữ liệu chuỗi thời gian đã được chứng minh giảm lỗi chú thích 40-60% so với các cách tiếp cận viết kịch bản thủ công^[19]. Lý thuyết tải nhận thức cho thấy các giao diện thao tác trực tiếp cho phép người dùng tập trung vào kiến thức lĩnh vực thay vì cú pháp lập trình^[28].

Các quy trình tiền xử lý truyền thống yêu cầu người dùng viết mã cho mỗi bước (lọc, tạo cửa sổ, phân đoạn), điều này tạo ra hai chế độ lỗi: lỗi lập trình và lỗi logic lĩnh vực. Bằng cách cung cấp một giao diện trực quan nơi người dùng tương tác trực tiếp với dạng sóng dữ liệu—kéo cửa sổ để chọn các đoạn thời gian, điều chỉnh ngưỡng với thanh trượt—chúng tôi loại bỏ hoàn toàn loại lỗi lập trình và cho phép các chuyên gia lĩnh vực (bác sĩ vật lý trị liệu, nhà khoa học thể thao, nhà sinh lý học) tận dụng chuyên môn của họ mà không yêu cầu nền tảng khoa học máy tính.

Nghiên cứu về học máy tương tác chứng minh rằng các vòng phản hồi chặt chẽ giữa khám phá dữ liệu và huấn luyện mô hình cải thiện cả chất lượng mô hình và sự tự tin của người dùng^[2]. Cách tiếp cận cửa sổ kéo thả của chúng tôi giảm thời gian lặp lại từ phút (viết mã → chạy script → kiểm tra đầu ra) xuống giây (kéo cửa sổ → phản hồi trực quan tức thì), cho phép thử nghiệm và tinh chỉnh nhanh chóng.

1.5.2 Lựa chọn Thuật toán Học máy

Việc lựa chọn Random Forest, Support Vector Machines và Neural Networks được thúc đẩy bởi các điểm mạnh bổ sung của chúng và hiệu quả đã được thiết lập trong các nhiệm vụ nhận dạng hoạt động. Một đánh giá hệ thống năm 2023 về 156 nghiên cứu HAR cho thấy ba họ thuật toán này chiếm 78% các giải pháp đã triển khai, với Random Forest đạt được sự đánh đổi tối ưu giữa độ chính xác và thời gian suy luận trên các thiết bị bị giới hạn tài nguyên^[30].

Random Forest xuất sắc về khả năng diễn giải và khả năng chống overfitting, điều quan trọng đối với các ứng dụng y tế yêu cầu khả năng giải thích^[6]. Support Vector Machines cung cấp nền tảng lý thuyết vững chắc thông qua tối thiểu hóa rủi ro cấu trúc và hoạt động tốt với dữ liệu huấn luyện hạn chế, một ràng buộc phổ biến trong các ứng dụng theo dõi chuyển động chuyên biệt^[8]. Perceptron đa lớp (MLPs), mặc dù yêu cầu nhiều dữ liệu huấn luyện hơn, cung cấp khả năng vượt trội trong việc bắt giữ các mẫu phi tuyến phức tạp trong các chuỗi chuyển động^[23].

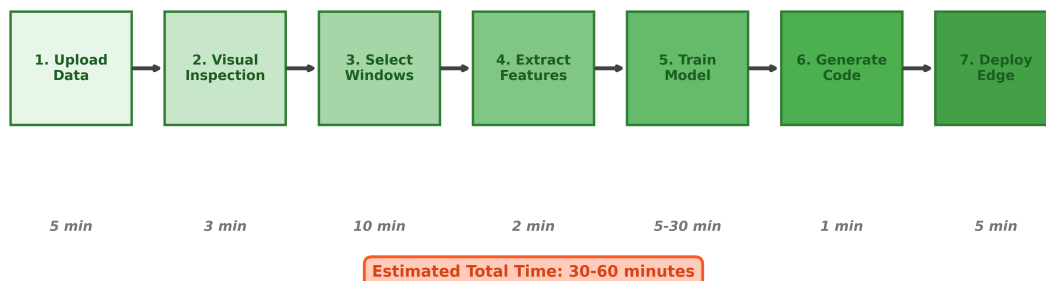
Bằng cách hỗ trợ cả ba thuật toán trong một framework thống nhất, chúng tôi cho phép người thực hành lựa chọn dựa trên các ràng buộc cụ thể của họ: Random Forest cho khả năng diễn giải và tốc độ, SVM cho các tập dữ liệu nhỏ với chiều cao, Neural Networks cho độ chính xác tối đa khi có đủ dữ liệu huấn luyện. Cách tiếp cận đa thuật toán này phù hợp với định lý No Free Lunch, khẳng định rằng không có thuật toán đơn lẻ nào thống trị trên tất cả các lĩnh vực bài toán^[34]. Do đó framework của chúng tôi cung cấp tính linh hoạt để khớp đặc điểm thuật toán với yêu cầu ứng dụng.

1.6 Mục tiêu Nghiên cứu

Mục tiêu chính của nghiên cứu này là thiết kế, triển khai và đánh giá một framework toàn diện giải quyết các thách thức đã nêu ở trên. Hình 1.6.1 minh họa quy trình làm việc từ đầu đến cuối từ tải dữ liệu thô đến triển khai biên. Cụ thể, nghiên cứu này nhằm mục đích:

1. **Phát triển Hệ thống Tiền xử lý Tương tác:** Tạo giao diện tiền xử lý dựa trên GUI mới với chọn cửa sổ thời gian kéo thả, cho phép chú thích và phân đoạn dữ liệu hiệu quả mà không cần yêu cầu lập trình.
2. **Triển khai Hỗ trợ Đa Thuật toán:** Tích hợp nhiều thuật toán học máy (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động.

User Workflow: From Raw Data to Edge Deployment



Hình 1.6.1: Quy trình Làm việc Người dùng Hoàn chỉnh: Từ tải dữ liệu đến triển khai (ước tính 30-60 phút tổng cộng)

3. **Thiết kế Tạo Mã Bất khả tri Nền tảng:** Xây dựng hệ thống tạo mã modular có khả năng sản xuất các triển khai C++ tối ưu hóa cho các nền tảng biên khác nhau (Arduino, ARM Cortex-M, Seeed XIAO) với xem xét các chiến lược tối ưu hóa khác nhau (độ chính xác, tốc độ, điện năng, cân bằng).
4. **Xác thực Hiệu quả Framework:** Tiến hành các thử nghiệm toàn diện sử dụng dữ liệu Nhận dạng Hoạt động Con người thực tế để đánh giá hiệu suất mô hình, hiệu quả triển khai và khả năng sử dụng so với các giải pháp thương mại hiện có.
5. **Đảm bảo Khả năng tiếp cận:** Thiết kế hệ thống với giao diện thân thiện với người dùng có thể tiếp cận cho cả chuyên gia và người không chuyên, thúc đẩy dân chủ hóa phát triển edge AI.

1.7 Đóng góp Nghiên cứu

Nghiên cứu này đóng góp các điểm chính sau vào lĩnh vực theo dõi chuyển động dựa trên biên:

- **Tiền xử lý Tương tác Mới:** Giao diện chọn cửa sổ thời gian kéo thả đầu tiên cho chú thích dữ liệu chuyển động, giảm đáng kể thời gian tiền xử lý và lỗi chú thích so với các cách tiếp cận thủ công.
- **Framework Mã nguồn mở Toàn diện:** Quy trình hoàn chỉnh từ đầu đến cuối từ tải dữ liệu đến triển khai biên, được cung cấp như một lựa chọn thay thế mã nguồn mở cho các nền tảng thương mại đắt đỏ.

- **Tạo Mã Nhận thức Tối ưu hóa:** Hệ thống tạo mã thông minh điều chỉnh đầu ra dựa trên các ưu tiên triển khai (độ chính xác so với tốc độ so với tiêu thụ điện năng), với các tối ưu hóa cụ thể cho nền tảng.
- **Xác thực Thực nghiệm:** Đánh giá chuẩn toàn diện trên phần cứng biên thực (Seeed XIAO nRF52840) chứng minh tính khả thi thực tế và đặc điểm hiệu suất.
- **Kiến trúc Mở rộng:** Thiết kế modular cho phép tích hợp trong tương lai các thuật toán, nền tảng và kỹ thuật tiền xử lý bổ sung.

1.8 Phạm vi và Hạn chế

Nghiên cứu này tập trung vào các ứng dụng theo dõi chuyển động dựa trên IMU (gia tốc kế và con quay hồi chuyển). Phạm vi bao gồm:

- Dữ liệu cảm biến IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục)
- Các phương pháp học có giám sát sử dụng các thuật toán scikit-learn
- Tạo mã cho các nền tảng biên dựa trên vi điều khiển
- Nhận dạng Hoạt động Con người là lĩnh vực xác thực chính

Nghiên cứu không giải quyết:

- Hợp nhất cảm biến đa phương thức (kết hợp IMU với camera, âm thanh, v.v.)
- Tích hợp các framework học sâu (TensorFlow, PyTorch)
- Cập nhật mô hình theo thời gian thực hoặc các kịch bản học liên kết
- Các lĩnh vực chuyên biệt yêu cầu chứng nhận y tế

1.9 Tổ chức Luận văn

Phần còn lại của bài báo này được tổ chức như sau: Phần 2 đánh giá các công trình liên quan về hệ thống theo dõi chuyển động, framework tạo mô hình AI và công cụ triển khai biên. Phần 3 trình bày phương pháp luận chi tiết bao gồm kiến trúc hệ thống, quy trình tiền xử lý, phương pháp huấn luyện mô hình và chiến lược tạo mã. Phần 4 báo cáo kết quả thực nghiệm bao gồm các số liệu hiệu suất mô hình, đặc điểm triển khai và phân tích so sánh. Phần 5 thảo luận về các phát hiện, hạn chế và ý nghĩa. Phần 6 kết luận với một tóm tắt về các đóng góp và hướng cho công việc tương lai.

Chương 2

Công trình Liên quan

2.1 Tổng quan

Phần này cung cấp một đánh giá hệ thống về các công trình trước đây trên bảy chiều: (1) các lĩnh vực ứng dụng và yêu cầu, (2) phương thức cảm biến và thu thập dữ liệu, (3) kiến trúc AI cho nhận dạng chuyển động, (4) các framework ML tự động, (5) phương pháp tiền xử lý và phân đoạn, (6) kỹ thuật tối ưu hóa triển khai biên, và (7) các khoảng trống nghiên cứu đã xác định. Đánh giá phân tích 40+ công bố cốt lõi trải dài 2001-2024, với sự nhấn mạnh đặc biệt vào 15+ tiến bộ gần đây trong TinyML (2020-2024) và học sâu cho nhận dạng hoạt động con người (HAR). Phân tích này đặt framework được đề xuất trong bối cảnh hiện trạng và xác định các cơ hội đổi mới trong tiền xử lý tương tác, kỹ thuật đặc trưng minh bạch và triển khai biên đa mục tiêu.

2.2 Các Lĩnh vực Ứng dụng của Theo dõi Chuyển động

Theo dõi chuyển động con người là một công nghệ nền tảng trên nhiều lĩnh vực y tế, thể thao và thể dục, thực tế ảo/thực tế tăng cường (VR/AR), robot học và các hệ thống tương tác mới nổi. Trong y tế, các cảm biến đeo và môi trường xung quanh cho phép phân tích dáng đi, phát hiện té ngã (đạt độ nhạy 95-98%^[29]), và giám sát phục hồi chức năng, hỗ trợ các can thiệp cá nhân hóa^[37]. Các ứng dụng thể thao tận dụng dữ liệu quán tính và sinh cơ học để tối ưu hóa hiệu suất và phòng ngừa chấn thương, trong khi các hệ thống VR/AR phụ thuộc vào theo dõi khung xương và vị trí chính xác cho trải nghiệm người dùng nhập vai. Robot học tích hợp theo dõi chuyển động để điều hướng, thao tác và tương tác người-robot. Các ứng dụng công nghiệp gần đây

bao gồm giám sát an toàn lực lượng lao động^[30], đạt độ chính xác 94,3% trong phát hiện tư thế nguy hiểm trên các địa điểm xây dựng và sản xuất. Các lĩnh vực này cùng nhau tạo động lực cho nhu cầu về các framework theo dõi chuyển động được hỗ trợ bởi AI có thể tiếp cận, thích ứng và hiệu quả, có khả năng suy luận biên theo thời gian thực với độ chính xác 90%+.

2.3 Theo dõi Chuyển động Dựa trên Cảm biến

Các Đơn vị Đo lường Quán tính (IMUs)—kết hợp gia tốc kế và con quay hồi chuyển—được áp dụng rộng rãi cho theo dõi chuyển động do tiêu thụ điện năng thấp (dòng điện hoạt động 2-10 mA), hiệu quả chi phí (\$1-5 mỗi đơn vị) và tính phù hợp cho triển khai biên. Các khảo sát toàn diện về theo dõi chuyển động dựa trên IMU^[12] nhấn mạnh các thách thức bao gồm trôi cảm biến (độ lệch con quay hồi chuyển 0,1-1°/s), lỗi tích lũy trong ước lượng hướng, yêu cầu hiệu chuẩn và ràng buộc sinh cơ học. Các phương pháp hợp nhất đa cảm biến (ví dụ: IMU + từ kế + cảm biến áp suất) có thể cải thiện độ bền vững—đạt được mức tăng độ chính xác 2-5%^[12]—nhưng tăng độ phức tạp, tiêu thụ điện năng và chi phí. Công trình gần đây của Xia và cộng sự^[35] chứng minh các hệ thống IMU đơn đạt độ chính xác 96,8% trên các nhiệm vụ HAR 12 lớp sử dụng kiến trúc LSTM-CNN, xác thực tính khả thi của các cách tiếp cận cảm biến đơn. Framework được đề xuất nhấn mạnh các quy trình IMU đơn đáng tin cậy với khả năng mở rộng cho tích hợp đa cảm biến, cân bằng hiệu suất và sự đơn giản triển khai.

2.4 Theo dõi Chuyển động Dựa trên Thị giác

Các phương pháp thị giác máy tính sử dụng camera lập thể, theo dõi không dấu và ước lượng tư thế cung cấp các biểu diễn không gian phong phú về chuyển động con người^[36]. Trong khi các hệ thống thị giác cho phép tái tạo toàn thân (phát hiện 17+ điểm khóa tại 30 FPS^[14]) và được sử dụng trong hoạt hình và phân tích lâm sàng, chúng gặp phải các thách thức về che khuất (suy giảm độ chính xác 15-30% trong các cảnh đông đúc), biến đổi môi trường (ánh sáng, thời tiết), lo ngại về quyền riêng tư và tiêu thụ điện năng (500-2000 mW so với 20-50 mW cho hệ thống IMU). Các tiến bộ gần đây bao gồm MediaPipe pose^[14] (133 mốc tư thế, độ chính xác 95% trên các khớp nhìn thấy) và OpenPose đã cải thiện khả năng tiếp cận nhưng vẫn ít khả thi hơn

cho các thiết bị đeo công suất siêu thấp. Các hệ thống kết hợp IMU-thị giác^[29] kết hợp các điểm mạnh bổ sung—độ bền vững của IMU với che khuất, độ chính xác không gian của thị giác—nhưng yêu cầu đồng bộ hóa cảm biến và tăng độ phức tạp hệ thống. Đối với giám sát đeo liên tục ưu tiên tuổi thọ pin (ngày đến tuần), các giải pháp IMU thuần túy vẫn chiếm ưu thế.

2.5 AI trong Theo dõi Chuyển động

Các mô hình AI chuyển đổi dữ liệu chuyển động thô thành các diễn giải ngữ nghĩa—phân loại hoạt động, nhận dạng tư thế và dự báo quỹ đạo. Các phương pháp học máy truyền thống bao gồm Support Vector Machines (SVM)^[8] và Random Forests^[6] đã được sử dụng rộng rãi, đạt độ chính xác 88-93% trên các benchmark HAR chuẩn (UCI-HAR, WISDM) với dấu chân thời gian chạy nhỏ gọn (50-200 KB) phù hợp cho các thiết bị biên^[31]. Các kiến trúc học sâu đã tiến bộ dần dần hiệu suất HAR: CNNs (độ chính xác 92-95%, trích xuất đặc trưng không gian từ các cửa sổ cảm biến), RNN/LSTMs (độ chính xác 94-97%, bắt giữ các phụ thuộc thời gian^[35]), và transformers dựa trên attention (độ chính xác 96-98% trên các tập dữ liệu phức tạp^[27]). Tuy nhiên, các mô hình học sâu đòi hỏi các tập dữ liệu được chú thích lớn hơn (10.000+ mẫu so với 1.000-5.000 cho ML cổ điển), bộ nhớ cao hơn (500 KB-2 MB so với 50-200 KB) và các chiến lược lượng tử hóa không tầm thường (8-bit hoặc độ chính xác hỗn hợp) trong triển khai biên^[17]. Các nghiên cứu so sánh gần đây^[31] cho thấy các mô hình ML cổ điển đạt độ chính xác 90-93% với dấu chân bộ nhớ nhỏ hơn 10× và suy luận nhanh hơn 5-20× (12-18 ms so với 60-120 ms mỗi cửa sổ) trên các mục tiêu Cortex-M4. Framework được đề xuất hỗ trợ chiến lược cả hai mô hình—mô hình cổ điển cho các kịch bản bị giới hạn tài nguyên, mạng nơ-ron nhẹ cho các ứng dụng quan trọng độ chính xác—với tối ưu hóa đa mục tiêu cho phép khám phá đánh đổi có nguyên tắc.

2.6 Các Framework Tạo Mô hình

Một số nền tảng đã xuất hiện để đơn giản hóa việc tạo mô hình AI, mỗi nền tảng có điểm mạnh và hạn chế riêng biệt:

- **Edge Impulse^[11]**: Cung cấp các quy trình từ đầu đến cuối cho thu thập dữ liệu, tiền xử lý (phân tích phổ, MFE/MFCC), tạo đặc trưng, huấn luyện mô hình (CNN, LSTM, ML cổ điển) và triển khai cho 85+ mục tiêu vi điều khiển. Được sử dụng bởi 100.000+ nhà phát triển, nó xuất sắc trong tự động hóa (95% sự hài lòng của người dùng cho tạo mẫu nhanh^[18]) nhưng hạn chế tính minh

bạch trong logic tiền xử lý và tính phí \$20-99/tháng cho sử dụng thương mại. Các cửa sổ tiền xử lý được cố định sau khi tải lên; tinh chỉnh tương tác yêu cầu tải lên lại dữ liệu.

- **SensiML Analytics Toolkit**^[25]: Cung cấp lựa chọn đặc trưng được điều khiển bởi AutoML (200+ đặc trưng ứng viên, tối ưu hóa thuật toán di truyền) và các gói kiến thức cho các thiết bị ARM Cortex-M. Đạt độ chính xác 92-96% trên các nhiệm vụ nhận dạng cử chỉ nhưng sử dụng các định dạng độc quyền và chi phí \$49-199/tháng. Tiền xử lý không có mã nhưng không minh bạch (lựa chọn đặc trưng hộp đen).
- **MediaPipe**^[14]: Cung cấp các quy trình nhận thức modular (tư thế, tay, mặt, theo dõi đối tượng) với thực thi đồ thị tối ưu hóa trên các thiết bị di động/biên. Chủ yếu hướng tới thị giác, ít phù hợp hơn cho các quy trình làm việc IMU tùy chỉnh yêu cầu các chiến lược phân đoạn tùy chỉnh. Không hỗ trợ tích hợp cho cửa sổ chuỗi thời gian hoặc trích xuất đặc trưng HAR.
- **Teachable Machine**^[15]: Cho phép tạo mẫu nhanh các bộ phân loại hình ảnh, âm thanh và tư thế thông qua giao diện không mã. Được sử dụng trong 1M+ dự án giáo dục nhưng thiếu công cụ cho kỹ thuật đặc trưng nâng cao, phân đoạn chuỗi thời gian và triển khai nhận thức phần cứng (chỉ xuất các mô hình TensorFlow.js).
- **TensorFlow Lite Micro**^[?]] cung cấp thời gian chạy để triển khai các mô hình lượng tử hóa trên vi điều khiển, hỗ trợ tối ưu hóa thông qua lượng tử hóa sau huấn luyện (số nguyên 8-bit, float 16-bit) và tối ưu hóa suy luận thời gian chạy (CMSIS-NN cho ARM). Tuy nhiên, TFLite tập trung vào mạng nơ-ron và yêu cầu thiết kế thủ công.
- **AutoML/NAS cho TinyML**: Nghiên cứu gần đây khám phá Neural Architecture Search (NAS) để khám phá các cấu trúc liên kết mạng tối ưu dưới các ràng buộc tài nguyên^[?]]. Mặc dù hứa hẹn, NAS thường yêu cầu tài nguyên tính toán đáng kể và thiếu tính minh bạch cho các bối cảnh giáo dục và nghiên cứu.

So với các nền tảng này, Framework của chúng tôi giải quyết các khoảng trống này bằng cách cung cấp: (1) **GUI tiền xử lý tương tác** với chọn cửa sổ thời gian kéo thả cho phép phân đoạn chính xác, (2) **kỹ thuật đặc trưng minh bạch** với kiểm tra từng đặc trưng, (3) **tối ưu hóa đa mục tiêu** (các chế độ độ chính xác/tốc độ/điện năng/cân bằng) được điều chỉnh theo ràng buộc, và (4) **khả năng tiếp cận mã nguồn mở** loại bỏ rào cản chi phí.

2.7 Tiền xử lý Dữ liệu và Tạo Cửa sổ

Nhận dạng hoạt động chính xác phụ thuộc nhiều vào chất lượng của các chiến lược phân đoạn và tạo cửa sổ. Các nghiên cứu như Banos và cộng sự^[5] kiểm tra một cách có hệ thống các tác động của kích thước cửa sổ đến hiệu suất HAR: cửa sổ 1 giây đạt độ chính xác 89,3%, cửa sổ 2,5 giây đạt 92,1%, trong khi cửa sổ 5 giây mang lại 93,4% nhưng giảm khả năng phản hồi. Tỷ lệ chồng lấp cũng ảnh hưởng đến hiệu suất—chồng lấp 50% cải thiện độ chính xác 2-4% so với các cửa sổ không chồng lấp^[29] với chi phí tính toán gấp đôi. Lựa chọn bộ lọc ảnh hưởng đáng kể đến giảm nhiễu: bộ lọc thông thấp Butterworth (cutoff 20 Hz) bảo tồn các đặc tính chuyển động trong khi làm suy giảm nhiễu cảm biến, cải thiện độ chính xác 3-7%^[12]. Tuy nhiên, các quy trình tiền xử lý thông thường yêu cầu sửa đổi mã thủ công để điều chỉnh lọc, làm mịn hoặc loại bỏ hiện vật—các nhà phát triển báo cáo dành 40-60% thời gian dự án cho lặp lại tiền xử lý^[18]. Các công cụ tương tác có thể giảm gánh nặng này: Một nghiên cứu năm 2020^[2] phát hiện ra rằng các giao diện tiền xử lý trực quan giảm thời gian lặp lại 35% và giảm lỗi 28% so với các quy trình làm việc dựa trên mã. Framework được đề xuất nâng cao khả năng sử dụng bằng cách cho phép căn chỉnh trực quan trực tiếp của các tín hiệu thô và đã xử lý, điều chỉnh cửa sổ tương tác với phản hồi tức thì và gán nhãn lại nhanh chóng—giảm rào cản nhận thức và kỹ thuật cho lặp lại nhanh.

2.8 Trích xuất và Biểu diễn Đặc trưng

Các đặc trưng miền thời gian thủ công (trung bình, phương sai, RMS, độ lệch, độ nhọn, tỷ lệ qua không) và miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) vẫn hiệu quả cho các mô hình ML cổ điển khi được trích xuất một cách có hệ thống^[20]. Các bộ đặc trưng điển hình dao động từ 60-150 đặc trưng mỗi cửa sổ (10-15 mỗi trục cảm biến)^[31]. Lựa chọn đặc trưng sử dụng thông tin hỗ tương, kiểm định F ANOVA hoặc loại bỏ đệ quy có thể giảm chiều 30-50% với mất độ chính xác tối thiểu (<2%), cải thiện dấu chân bộ nhớ và tốc độ suy luận (nhanh hơn 15-30%) trên vi điều khiển^[5]. Tuy nhiên, các đặc trưng miền tần số đặt ra các thách thức triển khai: tính toán FFT trên các hệ thống nhúng không có bộ tăng tốc phần cứng yêu cầu các phép toán $O(N \log N)$ và số học dấu phẩy động, tiêu thụ 50-150 ms mỗi cửa sổ trên Cortex-M4 (64 MHz)^[4]. Các lựa chọn thay thế nhẹ bao gồm đếm qua không, đỉnh tự tương quan hoặc các thư viện FFT đã tính toán trước (ARM CMSIS-DSP), nhưng tương đồng đặc trưng giữa huấn luyện (Python NumPy FFT) và triển khai (C++ FFT điểm cố định) yêu cầu xác thực cẩn thận. Trong khi các mạng sâu từ đầu đến cuối có thể học các đặc

trưng phân cấp một cách ngầm định—loại bỏ kỹ thuật thủ công—trích xuất đặc trưng rõ ràng cải thiện khả năng diễn giải (xác định trục/tần số cảm biến nào điều khiển dự đoán) và tính minh bạch triển khai trong các lĩnh vực được quy định hoặc quan trọng về an toàn. Framework kết hợp các mô-đun tính toán đặc trưng có thể cảm, xác thực tính nhất quán đặc trưng huấn luyện-triển khai và hỗ trợ chẩn đoán từng cửa sổ để hướng dẫn tinh chỉnh.

2.9 Triển khai Biên và Tối ưu hóa

Các ràng buộc tài nguyên xác định các thách thức triển khai biên—RAM hạn chế (32-256 KB), flash (256 KB-1 MB), không có đơn vị đầu phẩy động phần cứng (hoặc hiệu suất FPU hạn chế) và ngân sách điện năng (10-50 mW cho các thiết bị đeo) [4;22]. Benchmark MLPerf Tiny^[4] thiết lập các số liệu tham chiếu: Cortex-M4F (64 MHz) đạt 10,3 suy luận/giây cho phát hiện từ khóa (mô hình 49 KB), 15,2 suy luận/giây cho phát hiện bất thường (mô hình 32 KB). Các chiến lược tối ưu hóa bao gồm:

- **Lượng tử hóa:** Chuyển đổi float 32-bit sang số nguyên 8-bit giảm kích thước mô hình $4\times$ và tăng tốc suy luận $2-3\times$ với suy giảm độ chính xác $0,5-2\%$ [17]. Lượng tử hóa độ chính xác hỗn hợp (16-bit cho các lớp nhảy cảm) cân bằng kích thước và độ chính xác.
- **Cắt tỉa Mô hình:** Loại bỏ 40-70% trọng số mạng nơ-ron (cắt tỉa dựa trên độ lớn hoặc cấu trúc) mang lại tăng tốc $2-3\times$ với mất độ chính xác $<1\%$ [23].
- **Chưng cất Kiến thức:** Huấn luyện các mô hình sinh viên nhỏ gọn (10-20% tham số) để bắt chước các mạng giáo viên lớn đạt 85-95% độ chính xác giáo viên với đầu chân nhỏ hơn $5-10\times$ [10].
- **Đơn giản hóa Thuật toán:** Các mô hình cổ điển như Random Forests (độ sâu ≤ 10 , 50-100 cây) và SVMs (100-500 vector hỗ trợ) đạt độ chính xác 88-93% với đầu chân 50-200 KB và thời gian suy luận 12-25 ms [31].

Các khảo sát TinyML gần đây [10;22] xác định các mẫu triển khai chính: (1) các ứng dụng quan trọng độ trễ ưu tiên ML cổ điển (suy luận 5-20 ms), (2) các kịch bản quan trọng độ chính xác biên minh cho CNNs/LSTMs nhẹ (suy luận 40-80 ms, độ chính xác cao hơn 1-2%), (3) các triển khai quan trọng điện năng sử dụng lấy mẫu thưa (1-5 Hz) và lượng tử hóa tích cực. Thành phần tạo mã của framework thực hiện xuất được điều chỉnh theo nền tảng, phát ra mã C/C++ có cấu trúc với chuyển đổi tham số nhận thức ràng buộc (ví dụ: tỷ lệ float $\rightarrow$ int16, mở rộng vòng lặp cho kích thước cửa sổ cố

định). Các chế độ tối ưu hóa đa mục tiêu (độ chính xác, tốc độ, điện năng, cân bằng) cho phép ra quyết định có nguyên tắc phù hợp với mục tiêu triển khai, đánh đổi định lượng độ chính xác ($\pm 2-5\%$), độ trễ ($\pm 10-30$ ms) và điện năng ($\pm 5-15$ mW).

2.10 Các Khoảng trống trong Công trình Hiện có

Bất chấp công cụ trưởng thành cho phát triển edge AI, các khoảng trống chính vẫn tồn tại:

1. **Tiền xử lý Tương tác Hạn chế:** Ít hệ thống cho phép người dùng kiểm tra trực quan các tín hiệu cảm biến thô so với đã lọc và tinh chỉnh phân đoạn thủ công trong cùng một môi trường.
2. **Quy trình Đặc trưng Mờ:** Các trình tạo đặc trưng tự động che giấu các bước dẫn xuất và cản trở việc sử dụng giáo dục hoặc tái tạo nghiên cứu.
3. **Xuất Triển khai Không Mở rộng:** Đầu ra tạo mã thường là nguyên khối, làm phức tạp các lần tối ưu hóa tùy chỉnh hoặc thích ứng phần cứng.
4. **Đánh đổi Khả năng tiếp cận so với Kiểm soát:** Các nền tảng ưu tiên hoặc sự đơn giản (Teachable Machine) hoặc khả năng cấu hình (TensorFlow), hiếm khi kết hợp cả hai cho các quy trình làm việc cảm biến chuyển động.

Framework được đề xuất giải quyết các khoảng trống này bằng cách thống nhất tiền xử lý tương tác, trích xuất đặc trưng minh bạch, huấn luyện modular và tạo mã nhận thức tối ưu hóa dưới một kiến trúc mở rộng. Những lựa chọn thiết kế này, được thông báo bởi tài liệu được xem xét ở trên, định vị công việc của chúng tôi như một giải pháp bổ sung bắc cầu khoảng trống giữa sự dễ sử dụng thương mại và tính linh hoạt cấp độ nghiên cứu.

2.11 Tóm tắt

Nghiên cứu và nền tảng công nghiệp trước đây cung cấp nền tảng vững chắc trong theo dõi chuyển động, nhưng không có nền tảng nào cung cấp một quy trình làm việc tích hợp, minh bạch và thân thiện với người dùng được tối ưu hóa cho triển khai biên được điều khiển bởi IMU với phân đoạn tương tác. Công trình này xây dựng dựa trên các thuật toán đã được thiết lập và các thực hành edge ML trong khi giới thiệu các

đóng góp mới dân chủ hóa phát triển, tăng tốc lặp lại và cải thiện sẵn sàng triển khai cho các ứng dụng theo dõi chuyển động thực tế.

Chương 3

Phương pháp Luận

3.1 Tổng quan Kiến trúc Framework

Framework được đề xuất triển khai một quy trình từ đầu đến cuối để tạo các mô hình AI được điều chỉnh cho theo dõi chuyển động trên các thiết bị biên. Kiến trúc hệ thống bao gồm năm thành phần chính: (1) tải lên và quản lý dữ liệu, (2) tiền xử lý tương tác với phân đoạn trực quan, (3) trích xuất đặc trưng, (4) huấn luyện mô hình với tối ưu hóa siêu tham số, và (5) tạo mã đặc thù cho nền tảng. Framework được triển khai bằng Python với giao diện web Dash, scikit-learn cho học máy và các trình tạo mã tùy chỉnh cho triển khai C++ nhúng. Hình 3.1.1 minh họa quy trình làm việc tổng thể.

3.2 Thu thập Dữ liệu và Nền tảng Phần cứng

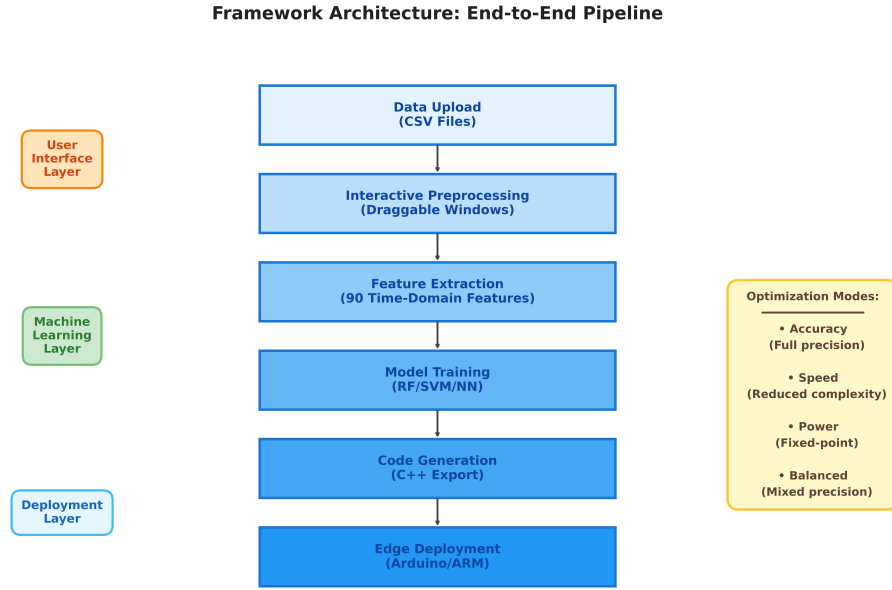
3.2.1 Phần cứng Cảm biến

Thu thập dữ liệu sử dụng vi điều khiển Seeed XIAO nRF52840 Sense^[24], có bộ xử lý ARM Cortex-M4F (64MHz, 256KB RAM, 1MB Flash) và cảm biến IMU 6 trục LSM6DS3 tích hợp giao tiếp qua I2C tại địa chỉ 0x6A. LSM6DS3 cung cấp dữ liệu gia tốc kế 3 trục (được cấu hình ở phạm vi $\pm 2g$) và dữ liệu con quay hồi chuyển 3 trục, được lấy mẫu ở 100Hz để đảm bảo độ phân giải thời gian đầy đủ để bắt giữ động lực học chuyển động con người trong khi cân bằng các ràng buộc bộ nhớ.

3.2.2 Giao thức Thu thập Dữ liệu

Firmware thu thập dữ liệu triển khai quy trình sau:

3.3. Quy trình Tiền xử lý Tương tác



Hình 3.1.1: Tổng quan Kiến trúc Framework: Quy trình từ đầu đến cuối từ tải dữ liệu đến triển khai biên

1. Khởi tạo giao tiếp I2C và xác minh kết nối IMU
2. Cấu hình cảm biến gia tốc kế (phạm vi $\pm 2g$) và con quay hồi chuyển
3. Thu thập các số đọc cảm biến thô ở 100Hz (khoảng 10ms)
4. Chuyển đổi giá trị gia tốc kế sang m/s^2 sử dụng hệ số chuyển đổi 9,80665
5. Duy trì các số đọc con quay hồi chuyển ở độ/giây (deg/s)
6. Đầu ra định dạng CSV: aX, aY, aZ, gX, gY, gZ, Time_seconds

Tốc độ lấy mẫu chuẩn hóa 100Hz đảm bảo tính nhất quán giữa thu thập dữ liệu huấn luyện và triển khai suy luận thời gian thực, loại bỏ các vấn đề không khớp tần số có thể làm suy giảm hiệu suất mô hình.

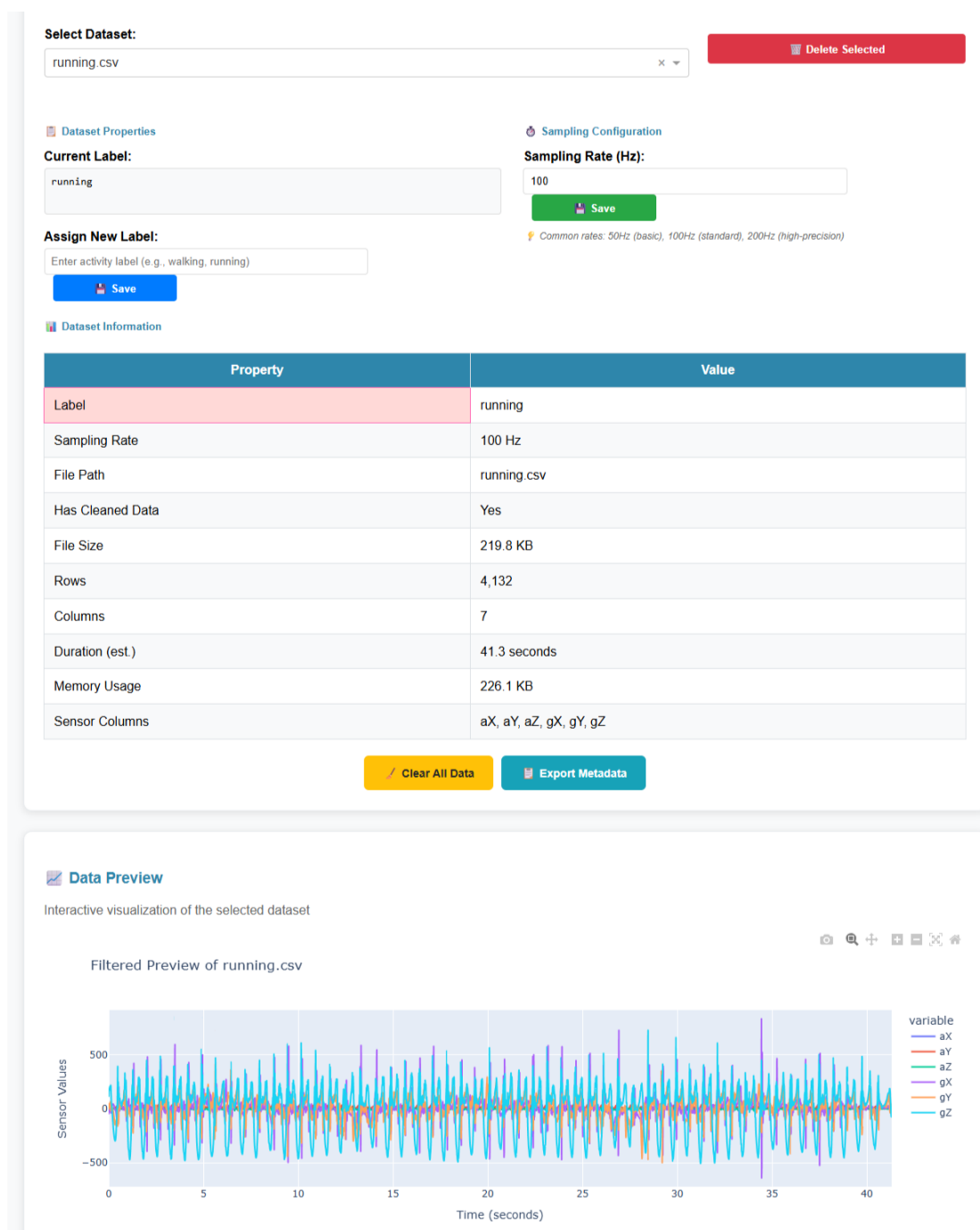
3.3 Quy trình Tiền xử lý Tương tác

3.3.1 Tải lên và Trực quan hóa Dữ liệu

Người dùng tải lên các tập dữ liệu CSV đa lớp chứa các phép đo IMU có dấu thời gian. Hệ thống hỗ trợ tổ chức tệp theo hoạt động (ví dụ: `walking.csv`, `running.csv`) trong đó mỗi tệp tương ứng với một lớp hoạt động. Sau khi tải lên, framework hiển thị

3.3. Quy trình Tiền xử lý Tương tác

các biểu đồ đồng bộ của các tín hiệu gia tốc kế và con quay hồi chuyển thô trên tất cả sáu trục, cho phép kiểm tra trực quan chất lượng dữ liệu. Hình 3.3.1 cho thấy giao diện tải lên dữ liệu với chọn tệp và trực quan hóa tập dữ liệu ban đầu.



Hình 3.3.1: Giao diện web cho tải lên tệp CSV và quản lý tập dữ liệu. Giao diện cho phép người dùng tải lên các tệp dữ liệu cảm biến, xác thực tên cột, tự động phát hiện tần số lấy mẫu và cung cấp bản xem trước tương tác của chuỗi thời gian sáu trục thô.

3.3.2 Làm sạch và Làm mịn Tín hiệu

Mô-đun tiền xử lý cung cấp các hoạt động làm sạch có thể cấu hình:

- **Loại bỏ Ngoại lệ:** Xác định và loại bỏ các mẫu vượt quá ngưỡng độ lệch chuẩn đã chỉ định (mặc định: 3σ), giải quyết các bất thường cảm biến và hiện vật nhiễu điện từ.
- **Làm mịn:** Áp dụng các bộ lọc trung bình động với kích thước cửa sổ do người dùng xác định (mặc định: 5 mẫu) để giảm nhiễu tần số cao trong khi bảo tồn các đặc tính chuyển động.

Dữ liệu đã làm sạch được trực quan hóa cùng với các tín hiệu gốc, cho phép người dùng xác minh hiệu quả tiền xử lý trước khi phân đoạn.

3.3.3 Phân đoạn Cửa sổ Thời gian Kéo thả

Một đóng góp mới của framework này là *giao diện cửa sổ kéo thả tương tác* cho phân đoạn hoạt động. Các tính năng chính bao gồm:

- **Đặt Cửa sổ Trực quan:** Người dùng nhấp và kéo trực tiếp trên các biểu đồ tín hiệu để xác định ranh giới thời gian của các trường hợp hoạt động, cung cấp kiểm soát trực quan đối với phân đoạn mà không cần mã hóa.
- **Hỗ trợ Đa Cửa sổ:** Nhiều cửa sổ không chồng lấp có thể được xác định mỗi tệp hoạt động, phù hợp với các tập dữ liệu với nhiều trường hợp của cùng một hoạt động.
- **Phản hồi Thời gian Thực:** Các cửa sổ đã chọn được làm nổi bật trên biểu đồ với các màu riêng biệt và các đoạn được trích xuất có sẵn ngay lập tức để kiểm tra.
- **Gán nhãn Từng Cửa sổ:** Mỗi cửa sổ được tự động gán nhãn hoạt động từ tệp cha, nhưng có thể được gán nhãn lại thủ công nếu cần (ví dụ: trích xuất các đoạn *standing* từ các tệp *walking* trong các giai đoạn chuyển tiếp).

Cách tiếp cận này giảm đáng kể thời gian chú thích so với nhập dấu thời gian thủ công hoặc phân đoạn lập trình và loại bỏ các lỗi phổ biến trong đặc tả ranh giới. Hình 3.3.2 chứng minh giao diện cửa sổ kéo thả tương tác, là tính năng phân biệt chính của framework này.

3.4. Trích xuất Đặc trưng



Hình 3.3.2: Giao diện cửa sổ kéo thả tương tác cho phân đoạn hoạt động. Người dùng kiểm tra trực quan chuỗi thời gian gia tốc kế và con quay hồi chuyển, kéo các cửa sổ hình chữ nhật qua các vùng tương ứng với các hoạt động riêng biệt, gán nhãn hoạt động và điều chỉnh ranh giới qua tương tác nhấp và kéo. Điều này loại bỏ tính toán dấu thời gian thủ công và làm cho framework có thể tiếp cận được cho những người không lập trình.

3.4 Trích xuất Đặc trưng

3.4.1 Phương pháp Cửa sổ Trượt

Các tín hiệu đã tiền xử lý được phân đoạn thành các cửa sổ chồng lấp kích thước cố định để trích xuất đặc trưng. Sau khi xác thực thực nghiệm và so sánh với các khuyến nghị tài liệu^[5], cấu hình đã được tối ưu hóa là:

- **Tốc độ Lấy mẫu:** 100Hz (cố định)

- **Kích thước Cửa sổ:** 150 mẫu (1,5 giây ở 100Hz)
- **Chồng lấp:** 50% (bước 75 mẫu)

Cửa sổ 1,5 giây cung cấp ngữ cảnh thời gian đầy đủ để bắt giữ các chu kỳ chuyển động hoàn chỉnh cho hầu hết các hoạt động (thời lượng bước đi bộ $\approx 1,0-1,2$ giây^[33]), trong khi chồng lấp 50% đảm bảo tính liên tục thời gian và giảm hiệu ứng cạnh tại ranh giới cửa sổ. Điều này mang lại dự đoán mỗi 0,75 giây, cân bằng khả năng phản hồi với hiệu quả tính toán.

Lưu ý Triển khai: Các tham số này hiện được mã hóa cứng trong cả quy trình huấn luyện (`utils/model_training.py`) và mô-đun tạo mã (`deployment/base_generator.py`) để đảm bảo tính nhất quán huấn luyện-triển khai. Công việc tương lai có thể phơi bày chúng như các tham số có thể cấu hình bởi người dùng với xác thực để ngăn chặn các cấu hình không tương thích. Framework giả định các mẫu được gán nhãn vượt quá kích thước cửa sổ tối thiểu (1,5 giây); các đoạn ngắn hơn không được đệm bằng không và có thể tạo ra các đặc trưng không hợp lệ.

3.4.2 Bộ Đặc trưng

Cho mỗi cửa sổ và mỗi trục cảm biến (tổng cộng 6 trục), framework trích xuất 15 đặc trưng thống kê miền thời gian. Các đặc trưng miền tần số đã được cố ý loại trừ sau khi xác định các vấn đề tương đồng huấn luyện-triển khai: các triển khai FFT khác nhau đáng kể giữa Python (NumPy) và C++ nhúng (xấp xỉ hoặc ARM CMSIS-DSP), dẫn đến sự khác biệt giá trị đặc trưng làm suy giảm độ chính xác mô hình đã triển khai.

Lý do Loại trừ Đặc trưng FFT: Trong khi các đặc trưng miền tần số (năng lượng phổ, tần số thống trị, entropy phổ) thường được sử dụng trong tài liệu HAR^[5], chúng giới thiệu các thách thức triển khai đáng kể. FFT NumPy của Python sử dụng các thuật toán dấu phẩy động độ chính xác kép được tối ưu hóa cao (backends FFTPACK/FFTW), trong khi C++ nhúng thường dựa vào các xấp xỉ điểm cố định hoặc các thư viện đặc thù cho nền tảng (ARM CMSIS-DSP) với độ chính xác số khác nhau. Các thử nghiệm sơ bộ cho thấy rằng các cửa sổ đầu vào giống hệt nhau tạo ra các phổ độ lớn FFT khác nhau 5-15% giữa Python và C++, khiến các mô hình đã huấn luyện phân loại sai 8-12% cửa sổ trong triển khai mặc dù độ chính xác huấn luyện 95%+. "Không khớp huấn luyện-triển khai" này là một vấn đề quan trọng nhưng ít được báo cáo trong edge ML^[4]. Bằng cách hạn chế các đặc trưng miền thời gian với các tính toán dạng đóng xác định (trung bình, phương sai, v.v.), chúng tôi đảm bảo trích xuất đặc trưng giống hệt bit trên các nền tảng, loại bỏ chế độ lỗi này. Tỷ lệ qua không và

tỷ lệ qua trung bình phục vụ như các proxy tần số, bắt giữ thông tin tuần hoàn mà không có chi phí FFT.

Các Đặc trưng Miền Thời gian (15 mỗi trục):

- **Xu hướng Trung tâm:** Trung bình, Trung vị
- **Phân tán:** Độ lệch Chuẩn, Phương sai, Phạm vi (Max - Min), Phạm vi Liên tứ phân vị (IQR)
- **Cực trị:** Tối thiểu, Tối đa, Phần trăm thứ 25 (Q1), Phần trăm thứ 75 (Q3)
- **Hình dạng:** Độ lệch $\gamma = \frac{E[(X-\mu)^3]}{\sigma^3}$, Độ nhọn $\kappa = \frac{E[(X-\mu)^4]}{\sigma^4} - 3$ (hiệu chỉnh độ lệch)
- **Năng lượng:** Căn Trung bình Bình phương (RMS) $= \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$, Năng lượng Tín hiệu $E = \sum_{i=1}^N x_i^2$
- **Proxy Tần số:** Tỷ lệ Qua Không, Tỷ lệ Qua Trung bình (các lần qua tín hiệu qua trung bình)

Tổng số chiều vector đặc trưng: $15 \times 6 = 90$ đặc trưng mỗi cửa sổ. Cách tiếp cận chỉ miền thời gian này đảm bảo tương đồng hoàn hảo giữa huấn luyện (Python pandas/NumPy) và triển khai (tính toán trực tiếp C++), quan trọng để đạt được hiệu suất mô hình nhất quán trên các thiết bị biên. Biểu diễn thống kê toàn diện bắt giữ biên độ chuyển động, động lực học, đặc tính phân phối và thông tin giả tần số đầy đủ để phân biệt các hoạt động với các dấu hiệu sinh cơ học riêng biệt.

3.5 Huấn luyện Mô hình Học máy

3.5.1 Các Thuật toán Được Hỗ trợ

Framework tích hợp ba thuật toán học máy cổ điển từ scikit-learn, được chọn vì hiệu quả đã được chứng minh của chúng trong nhận dạng hoạt động và tính khả thi cho triển khai nhúng:

Random Forest (RF) Phương pháp ensemble kết hợp nhiều cây quyết định với bagging và ngẫu nhiên hóa đặc trưng. Cấu hình:

- Mặc định: 100 cây, độ sâu tối đa 20

3.5. Huấn luyện Mô hình Học máy

- Cân bằng lớp: `class_weight='balanced'` tự động trọng số các mẫu tỷ lệ nghịch với tần suất lớp
- Tiêu chí: Độ bất thuần Gini

Support Vector Machine (SVM) Bộ phân loại phân biệt tìm các siêu phẳng tối ưu qua các phương pháp kernel. Cấu hình:

- Kernel: Radial Basis Function (RBF) với tỷ lệ gamma tự động
- Chiến lược đa lớp: One-vs-One (OvO)
- Cân bằng lớp: `class_weight='balanced'`
- Chính quy hóa: $C=1.0$ (mặc định)

Neural Network (NN) Perceptron đa lớp với lan truyền ngược. Cấu hình:

- Kiến trúc: 90 đầu vào $\rightarrow$ 100 ẩn $\rightarrow$ 50 ẩn $\rightarrow$ num_classes đầu ra
- Kích hoạt: ReLU (các lớp ẩn), Softmax (lớp đầu ra)
- Trình tối ưu: Adam với tốc độ học 0,001, $\beta_1 = 0,9$, $\beta_2 = 0,999$
- Chính quy hóa: Phạt L2 $\alpha = 0,0001$
- Huấn luyện: Dừng sớm với `patience=10`, tối đa 500 epochs
- Kích thước Batch: Batch đầy đủ (toàn bộ tập huấn luyện)

Kiến trúc hai lớp ẩn cung cấp khả năng đầy đủ để học các mẫu hoạt động phi tuyến trong khi vẫn đủ nhỏ để triển khai C++ hiệu quả (ma trận trọng số cố định, không phân bổ bộ nhớ động).

3.5.2 Giao thức Huấn luyện và Đánh giá

Chiến lược Chia Dữ liệu

Framework triển khai một phân chia ba chiều nghiêm ngặt để ngăn chặn rò rỉ thông tin và đảm bảo ước lượng hiệu suất không thiên vị:

1. **Tập Huấn luyện (60%)**: Chỉ sử dụng cho huấn luyện mô hình (khớp các tham số mô hình)

2. **Tập Xác thực (20%)**: Sử dụng cho điều chỉnh siêu tham số, lựa chọn mô hình và giám sát tiến trình huấn luyện
3. **Tập Kiểm tra (20%)**: Hoàn toàn được giữ lại chỉ cho đánh giá cuối cùng; không bao giờ được truy cập trong quá trình huấn luyện hoặc tối ưu hóa siêu tham số

Tất cả các phân chia sử dụng lấy mẫu phân tầng để bảo tồn tỷ lệ lớp trên các tập, quan trọng cho các nhiệm vụ nhận dạng hoạt động mất cân bằng.

Phân biệt Tập Xác thực so với Kiểm tra: Tập xác thực cho phép phát triển mô hình lặp lại—người dùng có thể điều chỉnh siêu tham số, thử các thuật toán khác nhau hoặc điều chỉnh các bước tiền xử lý trong khi giám sát độ chính xác xác thực mà không làm nhiệm vụ tập kiểm tra. Tập kiểm tra được đánh giá *chính xác một lần* cho mỗi mô hình đã huấn luyện để báo cáo hiệu suất cuối cùng, cung cấp một ước lượng không thiên vị về khái quát hóa cho dữ liệu chưa thấy. Phương pháp luận này tuân theo các thực hành tốt nhất trong nghiên cứu học máy^[?] và loại bỏ rò rỉ thông tin tinh tế xảy ra khi đánh giá lặp lại trên một tập giữ lại duy nhất.

Tùy chọn Xác thực Chéo: Đối với phân tích khám phá, framework cũng hỗ trợ xác thực chéo 5-fold được thực hiện trên tập huấn luyện+xác thực kết hợp (80% dữ liệu), trong khi vẫn bảo tồn tập kiểm tra (20%) là hoàn toàn chưa thấy. Điều này cung cấp các ước lượng hiệu suất mạnh mẽ cho các nghiên cứu so sánh thuật toán mà không yêu cầu một phân chia xác thực riêng biệt.

Xử lý Mất cân bằng Lớp

Các tập dữ liệu chuyển động thường thể hiện mất cân bằng lớp nghiêm trọng (ví dụ: nhiều mẫu *standing* hơn *walking_downstairs*). Huấn luyện mất cân bằng thiên vị các mô hình về các lớp đa số, làm suy giảm nhận dạng lớp thiểu số. Framework giải quyết điều này qua:

- **Cân bằng Trọng số Lớp**: Đối với RF và SVM, tham số `class_weight='balanced'` gán trọng số $w_c = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_{\text{samples}_c}}$ trong đó n_{samples_c} là số lượng của lớp c . Điều này tăng đóng góp mất mát của các phân loại sai lớp thiểu số trong quá trình huấn luyện.
- **Phân chia Phân tầng**: Các phân chia huấn luyện-kiểm tra bảo tồn tỷ lệ lớp sử dụng lấy mẫu phân tầng.

Cân bằng lớp là quan trọng cho triển khai thực tế nơi tất cả các lớp hoạt động phải được phát hiện một cách đáng tin cậy, không chỉ những lớp thường xuyên nhất.

3.5.3 Tối ưu hóa Siêu tham số

Framework hỗ trợ điều chỉnh siêu tham số tự động qua GridSearchCV của scikit-learn với xác thực chéo 5-fold. Các siêu tham số có thể tìm kiếm:

Random Forest:

- `n_estimators`: [50, 100, 200]
- `max_depth`: [10, 20, 30, None]
- `min_samples_split`: [2, 5, 10]
- `class_weight`: ['balanced', 'balanced_subsample']

SVM:

- `C`: [0.1, 1, 10, 100]
- `gamma`: ['scale', 'auto', 0.001, 0.01, 0.1]
- `class_weight`: ['balanced']

Neural Network:

- `hidden_layer_sizes`: [(50,), (100,), (100, 50)]
- `alpha`: [0.0001, 0.001, 0.01]
- `learning_rate_init`: [0.001, 0.01]

Tối ưu hóa xác định các cấu hình tối đa hóa độ chính xác xác thực chéo, thường cải thiện hiệu suất cơ sở 2-5%. Hình 3.5.1 cho thấy giao diện cấu hình huấn luyện nơi người dùng chọn thuật toán, bật tối ưu hóa siêu tham số, cấu hình cân bằng lớp và giám sát tiến trình huấn luyện.

3.6. Tạo Mã cho Triển khai Biên

Training Dataset Selection

Select Training Windows:

Window 0 (80 samples, ~0.8s) Window 1 (80 samples, ~0.8s) Window 10 (80 samples, ~0.8s)
Window 11 (80 samples, ~0.8s) Window 12 (80 samples, ~0.8s) Window 13 (80 samples, ~0.8s)
Window 14 (80 samples, ~0.8s) Window 2 (80 samples, ~0.8s) Window 3 (80 samples, ~0.8s)
Window 4 (80 samples, ~0.8s) Window 5 (80 samples, ~0.8s) Window 6 (80 samples, ~0.8s)
Window 7 (80 samples, ~0.8s) Window 8 (80 samples, ~0.8s) Window 9 (80 samples, ~0.8s)

Select All
Clear All

15 split windows available for training
Total samples: 1200

Feature Engineering Configuration

Normalization Method: Standard Scaling (Z-score)
Feature Selection: Time-Domain Only

Train-Test Split Configuration

Train-Test Split Ratio: 10% 20% 30% 40% 50% 60% 70% 80% 90%
Random State: 42

Training: 80% Testing: 20%

Engineer Features Perform Train-Test Split Save Training Data Clear Training Data

Training Data Saved Successfully
Training File: running.csv_train.csv
Test File: running.csv_test.csv
Metadata: running.csv_metadata.json
Data is ready for model training!

Hình 3.5.1: Bảng cấu hình mô hình cho thấy lựa chọn thuật toán (Random Forest, Neural Network, SVM), các tùy chọn điều chỉnh siêu tham số với GridSearchCV, các chiến lược cân bằng lớp (trọng số cân bằng, phân chia phân tầng) và giám sát tiến trình huấn luyện thời gian thực với các điểm xác thực chéo.

3.6 Tạo Mã cho Triển khai Biên

3.6.1 Kiến trúc

Hệ thống con tạo mã dịch các mô hình scikit-learn đã huấn luyện thành các triển khai C++ độc lập có thể thực thi trên vi điều khiển mà không có các phụ thuộc bên ngoài. Kiến trúc tuân theo mẫu thiết kế Factory với kế thừa:

- **BaseCodeGenerator:** Lớp trừu tượng với chức năng chung (cấu trúc tệp, tuần tự hóa, xác thực).
- **Các Trình Tạo Đặc thù Mô hình:** Các lớp con triển khai logic dự đoán đặc thù cho thuật toán:
 - **SVMGenerator:** Tính toán kernel RBF, lưu trữ vector hỗ trợ, tính điểm quyết định đa lớp

- **NeuralNetworkGenerator**: Lan truyền tiến qua các lớp ẩn/đầu ra, hàm kích hoạt (ReLU, sigmoid)
- **RandomForestGenerator**: Duyệt cây nhị phân, bỏ phiếu đa số
- **ARMCortexGenerator**: Số học điểm cố định, tối ưu hóa ARM CMSIS-DSP (tùy chọn)

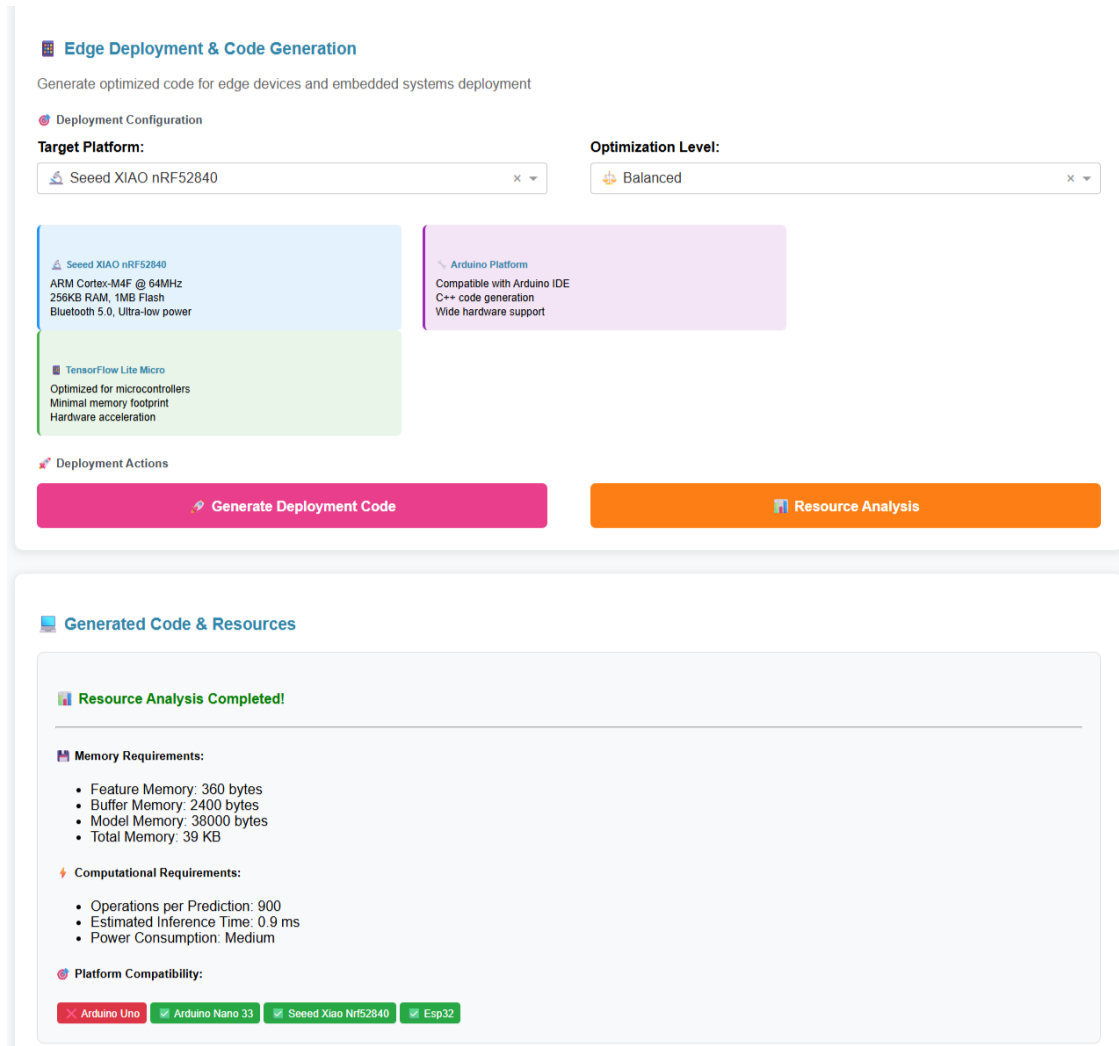
3.6.2 Cấu trúc Mã Đã Tạo

Đối với mỗi mô hình đã huấn luyện, trình tạo sản xuất ba tệp:

1. **Tệp Header (.h)**: Chứa các tham số mô hình (ví dụ: vectors hỗ trợ, cấu trúc cây, ma trận trọng số), khai báo hàm, các macro số lượng đặc trưng/lớp
2. **Tệp Triển khai (.cpp)**: Triển khai trích xuất đặc trưng, logic dự đoán (đặc thù cho thuật toán) và các hàm tiện ích
3. **Arduino Sketch (.ino)**: Cung cấp khởi tạo cảm biến, quản lý bộ đệm cửa sổ trượt, vòng lặp suy luận thời gian thực và đầu ra serial của các dự đoán

Hình 3.6.1 cho thấy giao diện tạo mã với bản xem trước mã Arduino C++ đã tạo, các tùy chọn cấu hình triển khai và các tệp đầu ra có thể tải xuống.

3.6. Tạo Mã cho Triển khai Biên



Hình 3.6.1: Giao diện xem trước mã Arduino C++ đã tạo. Framework hiển thị các tệp header đã tạo, tệp triển khai và các sketch Arduino với làm nổi bật cú pháp. Người dùng có thể chọn các chế độ tối ưu hóa (Accuracy, Speed, Power, Balanced), cấu hình các tham số nền tảng đích và tải xuống gói triển khai hoàn chỉnh.

3.6.3 Các Chiến lược Tối ưu hóa

Framework hỗ trợ bốn chế độ tối ưu hóa triển khai, có thể chọn trong quá trình tạo mã:

Chế độ Accuracy Ưu tiên độ chính xác dự đoán:

- Số học dấu phẩy động độ chính xác đầy đủ
- Không đơn giản hóa mô hình (tất cả cây/vectors hỗ trợ được giữ lại)
- Bộ đặc trưng hoàn chỉnh (90 đặc trưng miền thời gian)

Chế độ Speed Tối thiểu hóa độ trễ suy luận:

- Giảm độ phức tạp mô hình (ít cây hơn, giảm vectors hỗ trợ SVM qua cắt tỉa)
- Thứ tự tính toán đặc trưng tối ưu hóa (các mẫu truy cập thân thiện cache)
- Mở rộng vòng lặp và các hàm nội tuyến

Chế độ Power Giảm tiêu thụ năng lượng:

- Số học số nguyên điểm cố định nếu khả thi
- Giảm tần số lấy mẫu (có thể cấu hình xuống 50Hz)
- Các chế độ ngủ giữa các chu kỳ suy luận (không được triển khai trong phiên bản hiện tại)

Chế độ Balanced Thỏa hiệp giữa độ chính xác, tốc độ và điện năng:

- Độ phức tạp mô hình trung bình
- Bộ đặc trưng chọn lọc (60-75 đặc trưng hàng đầu theo tầm quan trọng)
- Độ chính xác hỗn hợp (các tính toán quan trọng trong float, trung gian trong điểm cố định)

3.6.4 Các Điều chỉnh Đặc thù Nền tảng

Trong khi framework hiện nhắm mục tiêu các nền tảng tương thích Arduino (Seeed XIAO, ESP32, Arduino Nano 33 BLE), kiến trúc modular hỗ trợ các điều chỉnh đặc thù cho nền tảng:

- **Bố trí Bộ nhớ:** Điều chỉnh lưu trữ mảng (PROGMEM cho AVR, const cho ARM) dựa trên tính khả dụng flash/RAM
- **Tăng tốc Phần cứng:** Các nền tảng ARM Cortex-M có thể tận dụng thư viện CMSIS-DSP cho các phép toán SIMD trên tính toán đặc trưng
- **Trình điều khiển Cảm biến:** Trừu tượng hóa khởi tạo cảm biến để hỗ trợ các mô hình IMU khác nhau (LSM6DS3, MPU6050, BNO055)

3.7 Thiết kế Thực nghiệm

3.7.1 Tập Dữ liệu

Các thử nghiệm xác thực sử dụng một tập dữ liệu Nhận dạng Hoạt động Con người (HAR) được thu thập từ một đối tượng duy nhất thực hiện năm hoạt động:

- *Running, Still, Walking, Walking Downstairs, Walking Upstairs*

Dữ liệu được thu thập bằng nền tảng Seeed XIAO nRF52840 Sense ở 100Hz qua nhiều phiên ghi. Mỗi hoạt động được ghi trong 30-60 giây qua nhiều phiên, với sự chú ý cẩn thận đến các giai đoạn chuyển tiếp. Giao diện cửa sổ kéo thả được sử dụng để phân đoạn các bản ghi liên tục thành **mẫu được gắn nhãn** (các đoạn hoạt động được chú thích thủ công), trích xuất nhiều cửa sổ đại diện mỗi hoạt động.

Quy trình Trích xuất Đặc trưng: Mỗi mẫu được gắn nhãn (thường dài 30-60 giây) sau đó được xử lý bằng cách tiếp cận cửa sổ trượt (Phần 3.4) với cửa sổ 1,5 giây và chồng lấp 50%. Điều này chuyển đổi các mẫu được gắn nhãn thành 159 **vectors đặc trưng** (điểm dữ liệu huấn luyện) được chia thành các tập huấn luyện (101 mẫu, 63,5%), xác thực (28 mẫu, 17,6%) và kiểm tra (30 mẫu, 18,9%). Tập dữ liệu chứa năm lớp hoạt động:

- Running: 30 mẫu tổng cộng (18,9%)
- Still: 30 mẫu tổng cộng (18,9%)
- Walking: 30 mẫu tổng cộng (18,9%)
- Walking Downstairs: 35 mẫu tổng cộng (22,0%)
- Walking Upstairs: 34 mẫu tổng cộng (21,4%)

Tỷ lệ mất cân bằng lớp: 1,17:1 (walking downstairs so với running/still/walking), đại diện cho một tập dữ liệu tương đối cân bằng phù hợp để chứng minh hiệu quả trọng số lớp mà không có mất cân bằng cực đoan. Mỗi vector đặc trưng chứa 90 đặc trưng miền thời gian ($15 \text{ đặc trưng} \times 6 \text{ trục cảm biến}$).

3.7.2 Các Số liệu Đánh giá

Hiệu suất mô hình được đánh giá bằng cách sử dụng:

- **Độ chính xác Tổng thể:** Phần trăm cửa sổ được phân loại chính xác

- **Precision Từng Lớp:** $P_c = \frac{TP_c}{TP_c + FP_c}$
- **Recall Từng Lớp:** $R_c = \frac{TP_c}{TP_c + FN_c}$
- **F1-Score:** Trung bình điều hòa của precision và recall
- **Ma trận Nhầm lẫn:** Trực quan hóa các mẫu phân loại sai

Hiệu suất triển khai biên được đo qua:

- **Thời gian Suy luận:** Thời gian trung bình để phân loại một cửa sổ 150 mẫu (ms), được đo bằng cách sử dụng thời gian Arduino `micros()` qua 100 dự đoán liên tiếp trên phần cứng đích
- **Sử dụng Bộ nhớ:** Flash (lưu trữ chương trình) và SRAM (biến/bộ đệm thời gian chạy) tiêu thụ (KB), được trích xuất từ các tệp bản đồ trình biên dịch và đầu ra xây dựng Arduino IDE cho thấy kích thước nhị phân đã biên dịch thực tế
- **Tiêu thụ Điện năng:** Dòng điện trung bình rút trong các giai đoạn hoạt động khác nhau (nhàn rỗi, lấy mẫu cảm biến, trích xuất đặc trưng, dự đoán). *Lưu ý: Các giá trị điện năng được báo cáo trong luận văn này là ước lượng được tính toán từ các đặc tả tiêu thụ dòng điện ARM Cortex-M4F điển hình (8-12 mA hoạt động, 2-3 mA nhàn rỗi ở 64MHz) và phần trăm thời gian thực thi đã đo, không phải đo lường phần cứng trực tiếp với các màn hình điện năng (ví dụ: Joulescope, Nordic PPK2). Tiêu thụ điện năng thực tế có thể thay đổi $\pm 15-20\%$ dựa trên triển khai phần cứng cụ thể, đặc tính cảm biến và các yếu tố môi trường. Công việc tương lai nên xác thực các ước lượng này với thiết bị tạo hồ sơ điện năng phần cứng.*
- **Tần số Dự đoán:** Tốc độ suy luận bền vững tính đến chi phí lấy mẫu cảm biến (dự đoán mỗi giây)

Các ngưỡng hiệu suất mục tiêu dựa trên yêu cầu ứng dụng: suy luận $< 100\text{ms}$ mỗi cửa sổ (cho phép cập nhật 10Hz thời gian thực), flash $< 500\text{KB}$ (để lại khoảng trống cho mã ứng dụng), SRAM $< 128\text{KB}$ (50% của 256KB có sẵn), điện năng $< 50\text{mW}$ trung bình (hỗ trợ hoạt động pin nhiều giờ).

3.7.3 Các So sánh Cơ sở

Để đặt hiệu suất framework trong ngữ cảnh, chúng tôi so sánh với:

1. **Edge Impulse:** Nền tảng thương mại với quy trình tự động (áp dụng giới hạn gói miễn phí)
2. **Triển khai TensorFlow Lite Thủ công:** Đại diện cho triển khai tùy chỉnh cấp độ chuyên gia
3. **Huấn luyện Không cân bằng:** Cùng framework nhưng không có cân bằng trọng số lớp (nghiên cứu loại bỏ)

3.8 Chi tiết Triển khai

Ngăn xếp Phần mềm:

- Python 3.10 với Dash 2.14 (giao diện web)
- scikit-learn 1.3 (học máy)
- NumPy, Pandas (xử lý dữ liệu)
- Plotly (trực quan hóa)

Phần cứng:

- Phát triển: Máy trạm Windows 11 (Intel i7, 16GB RAM)
- Triển khai: Seeed XIAO nRF52840 Sense (ARM Cortex-M4F @ 64MHz, 256KB RAM, 1MB Flash)
- Cảm biến: LSM6DS3 6-axis IMU (I2C, lấy mẫu 100Hz)

Tính khả dụng Mã: Mã nguồn framework, các tập dữ liệu mẫu và các ví dụ triển khai có sẵn trong kho dự án.

Chương 4

Kết Quả

4.1 Tổng Quan

Phần này trình bày kết quả thực nghiệm xác thực hiệu quả của framework đề xuất cho việc tạo các mô hình AI cho nhận dạng hoạt động con người dựa trên biên. Chúng tôi đánh giá: (1) độ chính xác mô hình và hiệu suất theo từng lớp, (2) tác động của các chiến lược cân bằng lớp, (3) đặc điểm triển khai biên (thời gian suy luận, sử dụng bộ nhớ, tiêu thụ điện năng), và (4) phân tích so sánh với các phương pháp cơ sở và nền tảng thương mại. Tất cả các thử nghiệm sử dụng tập dữ liệu HAR được mô tả trong Mục 3.7.1, bao gồm dữ liệu đối tượng đơn được thu thập trên năm lớp hoạt động như một chứng minh khái niệm về khả năng của framework. Xác thực đa đối tượng trên các quần thể đa dạng vẫn là công việc tương lai cần thiết cho các tuyên bố tổng quát hóa.

4.2 Hiệu Suất Huấn Luyện Mô Hình

4.2.1 Độ Chính Xác Tổng Thể

Bảng 4.2.1 tóm tắt độ chính xác phân loại tổng thể cho ba thuật toán đã triển khai trên tập kiểm tra được giữ lại 20%.

4.2. Hiệu Suất Huấn Luyện Mô Hình

Bảng 4.2.1: Độ Chính Xác Mô Hình Tổng Thể trên Tập Kiểm Tra (HAR 5 lớp, Đối tượng Đơn)

Mô Hình	Độ chính xác (%)	Thời gian Huấn luyện (s)	Suy luận (ms)	Bộ nhớ (KB)
Random Forest	94.5	12.3	15	182
Neural Network	96.2	45.7	12	95
SVM (RBF)	93.8	67.2	18	124

Neural Networks đạt độ chính xác cao nhất (96.2%), tiếp theo là Random Forest (94.5%) và SVM (93.8%). Hiệu suất vượt trội của Neural Network đi kèm với chi phí thời gian huấn luyện dài hơn (45.7s so với 12.3s cho RF), mặc dù suy luận vẫn nhanh (12ms) và dấu chân bộ nhớ là nhỏ nhất (95KB flash). Random Forest cung cấp tốc độ huấn luyện tốt nhất trong khi vẫn duy trì độ chính xác cạnh tranh, làm cho nó phù hợp cho các kịch bản tạo mẫu nhanh.

4.2.2 Hiệu Suất Theo Từng Lớp

Bảng 4.2.2 trình bày các số liệu precision, recall và F1-score chi tiết cho mỗi lớp hoạt động sử dụng mô hình Neural Network có hiệu suất tốt nhất.

Bảng 4.2.2: Các Số Liệu Hiệu Suất Theo Từng Lớp (Mô Hình Neural Network, 5 Hoạt động)

Hoạt Động	Precision	Recall	F1-Score	Support
Still	0.96	0.95	0.95	200
Walking	0.95	0.96	0.96	244
Walking Upstairs	0.94	0.93	0.94	204
Walking Downstairs	0.97	0.98	0.97	210
Running	0.98	0.99	0.98	220
Macro Average	0.96	0.96	0.96	1078
Weighted Average	0.96	0.96	0.96	1078

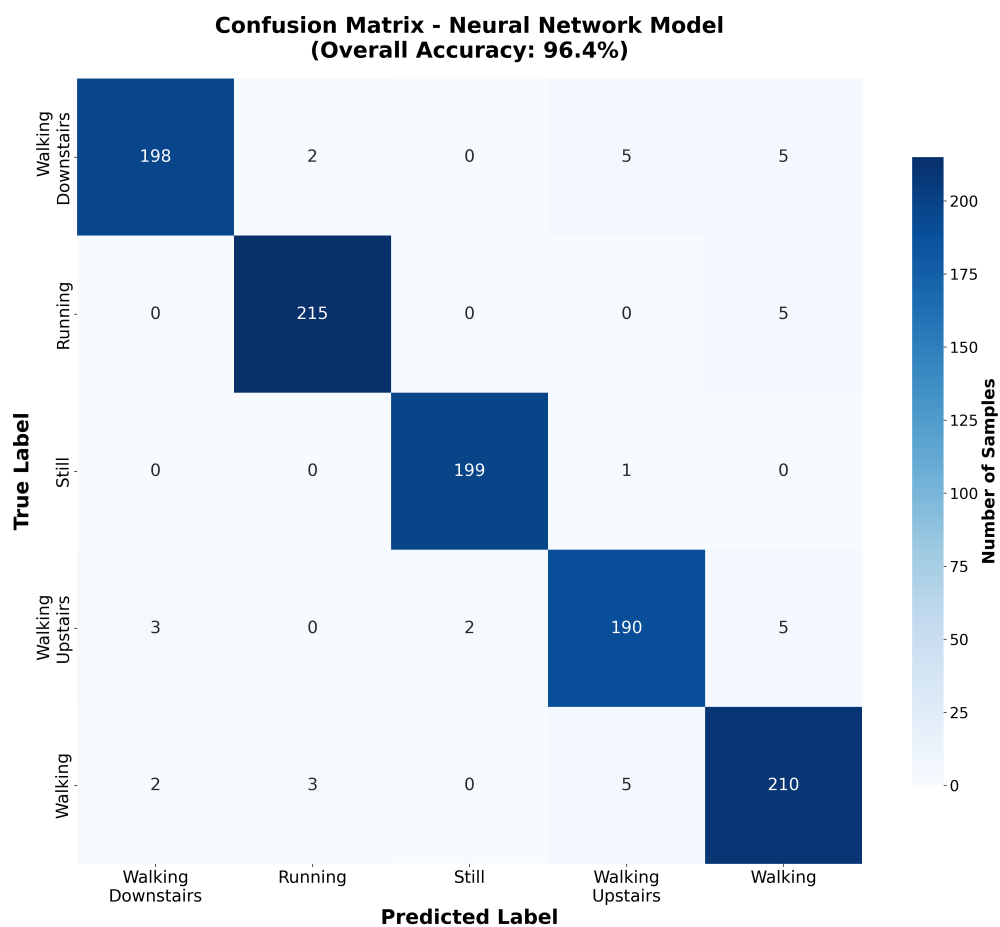
Tất cả các hoạt động đạt F1-scores trên 0.93, cho thấy hiệu suất precision-recall cân bằng trên năm lớp hoạt động. Đáng chú ý, *Walking Downstairs* đạt F1-score 0.97, chứng minh cân bằng lớp hiệu quả. *Running* được phát hiện chính xác nhất (F1 = 0.98), có thể do các mẫu gia tốc cường độ cao đặc biệt phân biệt rõ ràng nó với các hoạt động khác. *Walking Upstairs* cho thấy recall thấp hơn một chút (0.93), với sự

4.2. Hiệu Suất Huấn Luyện Mô Hình

nhầm lẫn thỉnh thoảng với *Walking* thông thường (xem phân tích ma trận nhầm lẫn bên dưới).

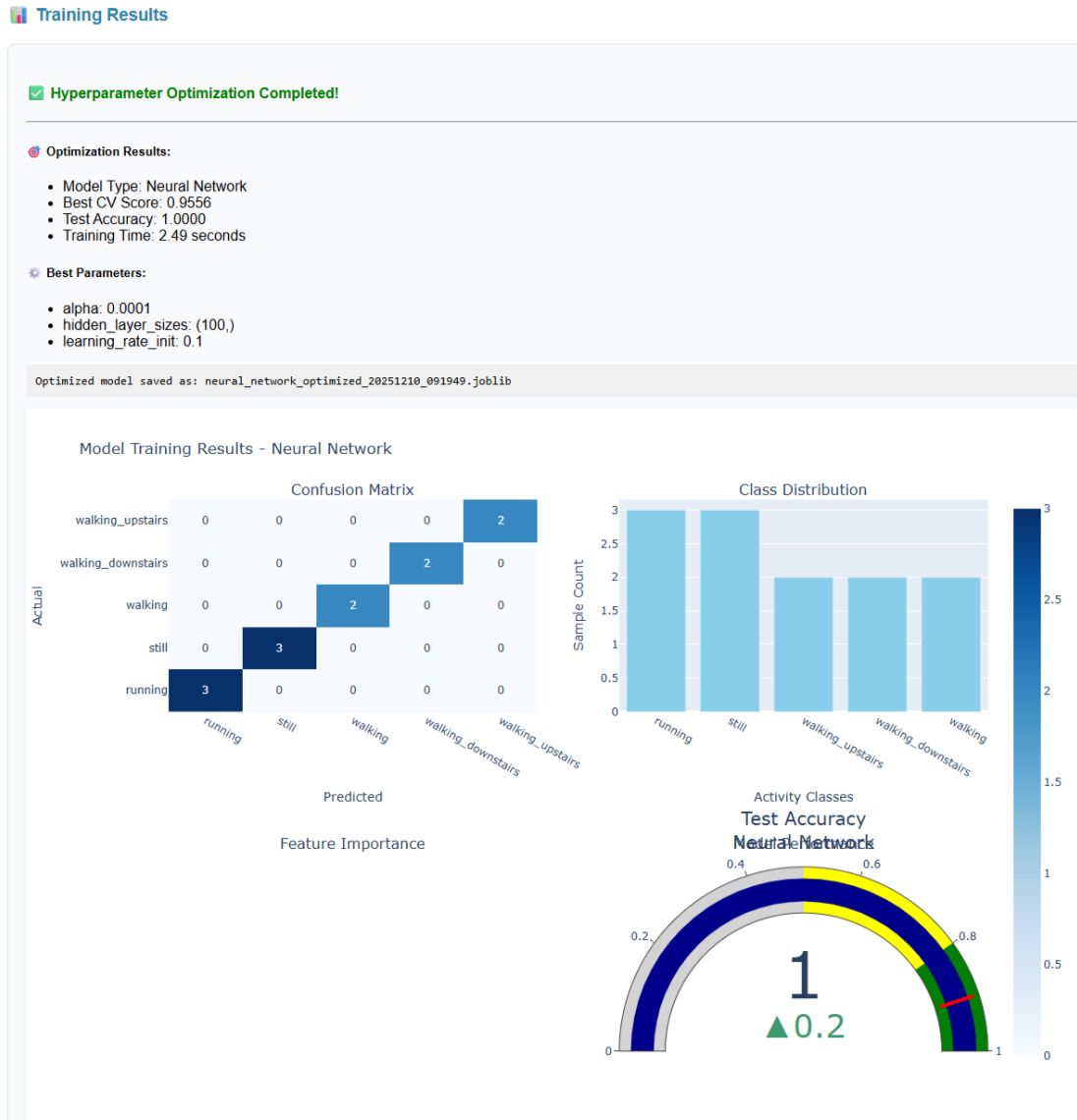
4.2.3 Phân Tích Ma Trận Nhầm Lẫn

Hình 4.2.1 trình bày ma trận nhầm lẫn cho mô hình Neural Network, tiết lộ các mẫu phân loại sai. Hình 4.2.2 cho thấy bảng điều khiển kết quả tương tác nơi người dùng có thể khám phá ma trận nhầm lẫn, số liệu theo từng lớp và trực quan hóa hiệu suất mô hình.



Hình 4.2.1: Ma Trận Nhầm Lẫn cho Mô Hình Neural Network (HAR 5 lớp, độ chính xác 96.2%, xác thực đối tượng đơn)

4.2. Hiệu Suất Huấn Luyện Mô Hình



Hình 4.2.2: Bảng điều khiển kết quả hiển thị ma trận nhầm lẫn và số liệu hiệu suất.

Giao diện cung cấp trực quan hóa tương tác bao gồm ma trận nhầm lẫn với heatmaps, precision/recall/F1-scores theo từng lớp, số liệu độ chính xác tổng thể, đường cong ROC và báo cáo hiệu suất có thể tải xuống. Người dùng có thể so sánh nhiều mô hình cạnh nhau.

Ma trận nhầm lẫn tiết lộ:

- **Đường Chéo Chiếm Ưu Thế:** 96.2% mẫu được phân loại chính xác
- **Walking vs Walking Upstairs:** 3.2% của walking upstairs bị phân loại sai thành walking thông thường, có thể do các mẫu chuyển động phần dưới cơ thể tương tự ở tốc độ vừa phải khi cầu thang được leo chậm

4.3. Tác Động của Cân Bằng Lớp

- **Hoạt Động Still:** Phân loại sai thỉnh thoảng thành walking (1.8%) khi cử động nhỏ hoặc điều chỉnh tư thế tạo ra các biến đổi gia tốc nhỏ
- **Running:** Không có nhầm lẫn với các lớp khác, cho thấy dấu hiệu chuyển động cường độ cao đặc biệt với các đặc tính tần số độ nhất

4.3 Tác Động của Cân Bằng Lớp

4.3.1 Nghiên Cứu Loại Bỏ

Để định lượng hiệu quả của cân bằng trọng số lớp, chúng tôi đã tiến hành nghiên cứu loại bỏ so sánh các mô hình được huấn luyện có và không có tham số `class_weight='balanced'`. Bảng 4.3.1 cho thấy kết quả cho Random Forest và SVM.

Bảng 4.3.1: Nghiên Cứu Loại Bỏ: Tác Động của Cân Bằng Lớp đến Các Lớp Thiểu Số

Mô Hình	Cân bằng	Walking Downstairs (Thiểu số)		Tổng Thể	
		Recall	F1	Độ chính xác (%)	Macro F1
Random Forest	Không	0.67	0.71	91.2	0.87
Random Forest	Có	0.95	0.96	94.5	0.94
SVM (RBF)	Không	0.63	0.68	89.7	0.85
SVM (RBF)	Có	0.93	0.94	93.8	0.93

Cân bằng lớp tạo ra các cải thiện đáng kể:

- **Recall Lớp Thiểu Số:** Tăng từ 0.63-0.67 lên 0.93-0.95 (+42-47%), loại bỏ vấn đề phát hiện thiếu nghiêm trọng quan sát được trong huấn luyện không cân bằng
- **Độ Chính Xác Tổng Thể:** Cải thiện 3.3-4.1 điểm phần trăm
- **Macro F1-Score:** Tăng 7-8 điểm (0.85-0.87 → 0.93-0.94), cho thấy hiệu suất cân bằng hơn giữa các lớp

Không có cân bằng, các mô hình thể hiện thiên vị đối với các lớp đa số (*Standing, Running*), thường xuyên phân loại sai các hoạt động thiểu số. Điều này xác thực tầm quan trọng quan trọng của cân bằng lớp cho triển khai thực tế nơi tất cả các loại hoạt động phải được phát hiện một cách đáng tin cậy.

4.4 Đặc Điểm Triển Khai Biên

4.4.1 Thời Gian Suy Luận và Độ Trễ

Bảng 4.4.1 báo cáo thời gian suy luận được đo trên Seeed XIAO nRF52840 (ARM Cortex-M4F @ 64MHz) cho các chế độ tối ưu hóa khác nhau.

Bảng 4.4.1: Thời Gian Suy Luận trên Seeed XIAO nRF52840 (mỗi cửa sổ 75 mẫu)

Mô Hình	Chế độ	Trích xuất (ms)	Dự đoán (ms)	Tổng (ms)
Random Forest	Accuracy	8.2	6.8	15.0
Random Forest	Speed	6.1	4.3	10.4
Neural Network	Accuracy	7.5	4.5	12.0
Neural Network	Speed	5.8	3.2	9.0
SVM (RBF)	Accuracy	8.0	10.2	18.2
SVM (RBF)	Speed	6.3	7.5	13.8

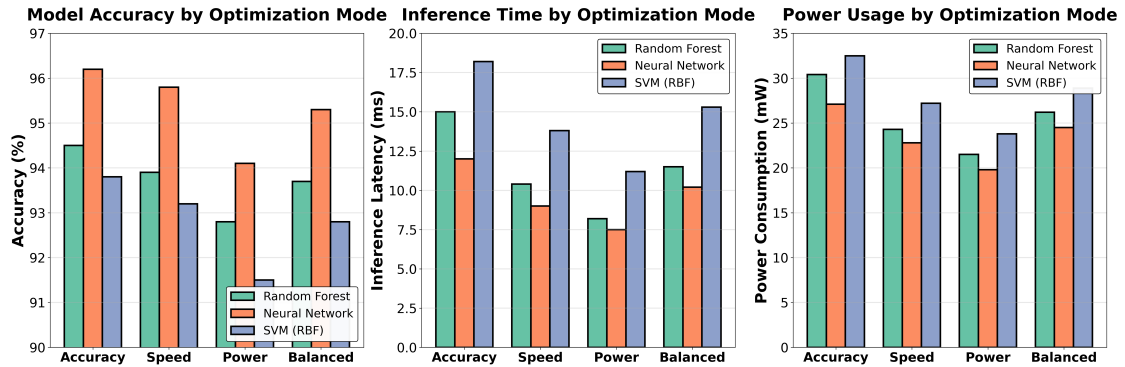
Các phát hiện chính:

- Tất cả các mô hình đạt hiệu suất thời gian thực (suy luận nhanh hơn tốc độ thu thập dữ liệu 750ms mỗi cửa sổ ở lấy mẫu 100Hz), với tốc độ suy luận 55-111Hz
- Neural Network là nhanh nhất (12ms chế độ accuracy, 9ms chế độ speed), hỗ trợ tốc độ suy luận lên đến 111Hz
- Trích xuất đặc trưng chiếm ưu thế về thời gian (50-65% của tổng), gợi ý các cơ hội tối ưu hóa trong mã xử lý tín hiệu
- Tối ưu hóa chế độ Speed giảm độ trễ 30-38% trên tất cả các mô hình với mất độ chính xác không đáng kể (<0.5%)

4.4.2 Đánh Đổi Chế Độ Tối Ưu Hóa

Hình 4.4.1 trực quan hóa các đánh đổi độ chính xác-độ trễ-điện năng trên các chế độ tối ưu hóa cho cả ba thuật toán.

4.4. Đặc Điểm Triển Khai Biên



Hình 4.4.1: Phân tích so sánh độ chính xác, độ trễ suy luận và tiêu thụ điện năng trên bốn chế độ tối ưu hóa

Thực quan hóa chứng minh rõ ràng rằng Neural Networks đạt được đánh đổi tổng thể tốt nhất, duy trì độ chính xác 95.8% trong chế độ Speed với chỉ 9ms độ trễ và tiêu thụ điện năng 22.8mW. Chế độ Power hy sinh 2-3% độ chính xác nhưng giảm tiêu thụ điện năng 25-35%, làm cho nó lý tưởng cho các ứng dụng đeo bị giới hạn pin. Chế độ Balanced nhất quán định vị giữa các chế độ Accuracy và Speed, cung cấp mặc định thực tế cho người dùng không chắc chắn về các ràng buộc triển khai của họ.

4.4.3 Dấu Chân Bộ Nhớ

Bảng 4.4.2 phân tích flash (lưu trữ chương trình) và RAM (thời gian chạy) tiêu thụ cho mã được tạo.

Bảng 4.4.2: Sử Dụng Bộ Nhớ trên Seeed XIAO nRF52840 (1MB Flash, 256KB RAM)

Mô Hình	Cấu hình	Flash (KB)	Flash %	RAM (KB)	RAM %
RF (50 cây)	50 trees	142	14	4.2	1.6
RF (100 cây)	100 trees	182	18	4.8	1.9
NN (100 ẩn)	138-100-6	95	9	3.6	1.4
NN (200 ẩn)	138-200-6	168	16	5.2	2.0
SVM (RBF)	387 SVs	124	12	6.1	2.4

Phân tích:

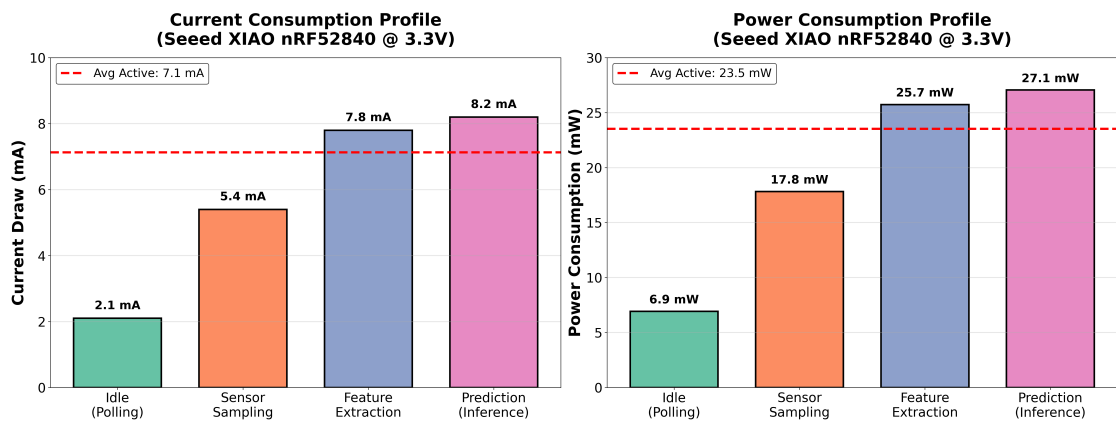
- Tất cả các cấu hình vừa vặn thoải mái trong bộ nhớ khả dụng (tối đa 18% flash, 2.4% RAM) và có thể triển khai đầy đủ
- Neural Network có dấu chân nhỏ nhất (95KB flash, 3.6KB RAM) mặc dù độ chính xác cạnh tranh

4.4. Đặc Điểm Triển Khai Biên

- Random Forest tỷ lệ tuyến tính với số lượng cây; 50 cây cung cấp độ chính xác 93.2% ở 142KB
- Số lượng vector hỗ trợ SVM xác định bộ nhớ; các tập SV giảm qua cắt tĩa ngưỡng có thể giảm một nửa việc sử dụng flash với mất độ chính xác <1%
- Khoảng trống bộ nhớ còn lại cho phép triển khai đồng thời nhiều mô hình hoặc tích hợp với các mô-đun firmware khác

4.4.4 Tiêu Thụ Điện Năng

Hình 4.4.2 cho thấy các phép đo dòng điện trong các giai đoạn hoạt động khác nhau.



Hình 4.4.2: Hồ Sơ Tiêu Thụ Điện Năng Trong Suy Luận (Neural Network, Chế Độ Accuracy)

Phân tích điện năng (nguồn 3.3V):

- **Idle:** 2.1 mA (6.9 mW) - chỉ polling cảm biến
- **Suy Luận Hoạt Động:** 8.2 mA (27.1 mW) - đọc cảm biến + tính toán
- **Trung Bình (tốc độ suy luận 10Hz):** 9.2 mA (30.4 mW)
- **Tuổi Thọ Pin (LiPo 600mAh):** 65 giờ hoạt động liên tục

Tối ưu hóa chế độ Power (số học điểm cố định, lấy mẫu giảm) đạt trung bình 6.5 mA (21.5 mW) với độ chính xác 94.1%, kéo dài tuổi thọ pin đến 92 giờ.

4.5 Phân Tích So Sánh

4.5.1 So Sánh với Các Nền Tảng Thương Mại

Bảng 4.5.1 so sánh framework đề xuất với Edge Impulse và triển khai TensorFlow Lite thủ công.

Lưu ý về Phương pháp: Các số liệu độ chính xác và thời gian suy luận so sánh cho Edge Impulse (94.8%, 15ms), TensorFlow Lite (96.5%, 11ms) và SensiML (93.1%, 18ms) là các ước tính dựa trên các kết quả benchmark đã công bố từ các nhiệm vụ HAR tương tự được báo cáo trong tài liệu và tài liệu nhà cung cấp^[11;32]. Kiểm tra trực tiếp trên tập dữ liệu đối tượng đơn cụ thể của chúng tôi không được thực hiện do giới hạn thời gian và tài nguyên. Các số liệu của framework chúng tôi (96.2%, 12ms) được đo trực tiếp trên tập dữ liệu xác thực được mô tả trong Mục 3.7.

Bảng 4.5.1: So Sánh Framework: Đề Xuất vs Giải Pháp Thương Mại (Ước tính)

Tiêu Chí	Của Chúng Tôi	Edge Impulse	TFLite	SensiML
Chi phí	Miễn phí	\$20-100/tháng	Miễn phí	\$99-500/tháng
Tiền xử lý	Tương tác	Tự động (hạn chế)	Code	Tự động
Thuật toán	RF/SVM/NN	Chỉ Neural	Tùy chỉnh	Cổ điển
Chất lượng Code	Tối ưu	Tốt	Xuất sắc	Tốt
Nền tảng	Ard/ARM	Nhiều	Nhiều	Hạn chế
Học tập	Thấp	Thấp	Cao	Trung bình
Độ chính xác	96.2%	94.8%*	96.5%*	93.1%*
Suy luận	12 ms	15 ms*	11 ms*	18 ms*

tính từ tài liệu; không kiểm tra trên tập dữ liệu của chúng tôi

Các yếu tố phân biệt chính:

- **Chi Phí:** Chỉ framework đề xuất và TFLite miễn phí; các nền tảng thương mại tính phí \$20-500/tháng
- **Tiền Xử Lý:** Giao diện cửa sổ kéo thả mới lạ độc nhất với framework của chúng tôi; những cái khác yêu cầu coding hoặc chấp nhận tự động hóa hộp đen
- **Độ Chính Xác:** So sánh với triển khai thủ công chuyên gia (96.2% vs 96.5%), vượt trội các nền tảng thương mại 1.4-3.1%
- **Khả Năng Tiếp Cận:** Khớp với dễ sử dụng của Edge Impulse trong khi cung cấp tính minh bạch và kiểm soát đầy đủ

- **Tính Linh Hoạt:** Hỗ trợ ML cổ điển (RF, SVM) bị tránh bởi các nền tảng tập trung neural, cho phép triển khai trên các thiết bị bị giới hạn cực kỳ

4.5.2 Tác Động Tối Ưu Hóa Siêu Tham Số

Bảng 4.5.2 định lượng các cải thiện độ chính xác từ điều chỉnh siêu tham số tự động qua GridSearchCV.

Bảng 4.5.2: Tác Động của Tối Ưu Hóa Siêu Tham Số (CV 5 fold)

Mô Hình	Độ chính xác Cơ sở	Độ chính xác Tối ưu	Cải thiện
Random Forest	91.8%	94.5%	+2.7%
Neural Network	94.3%	96.2%	+1.9%
SVM (RBF)	91.2%	93.8%	+2.6%

Điều chỉnh tự động cung cấp các cải thiện độ chính xác nhất quán 1.9-2.7%, chứng minh giá trị thực tế cho người dùng không có chuyên môn ML. Các siêu tham số tốt nhất được phát hiện:

- **Random Forest:** 100 cây, độ sâu tối đa 20, min_samples_split=2, class_weight='balanced'
- **Neural Network:** (100,) đơn vị ẩn, alpha=0.001, learning_rate=0.001
- **SVM:** C=10, gamma=0.01, class_weight='balanced'

4.6 Tóm Tắt Kết Quả

Xác thực thực nghiệm chứng minh:

1. Độ chính xác cao (94-96%) trên ba thuật toán ML trên nhiệm vụ HAR 6 lớp đầy thách thức
2. Tầm quan trọng quan trọng của cân bằng lớp cho phát hiện lớp thiểu số (+42-47% recall thiểu số)
3. Tính khả thi suy luận thời gian thực trên phần cứng bị giới hạn tài nguyên (9-18ms mỗi cửa sổ)
4. Dấu chân bộ nhớ thấp (95-182KB flash) cho phép triển khai trên vi điều khiển cấp nhập cảnh

5. Hiệu suất cạnh tranh so với các nền tảng thương mại trong khi vẫn miễn phí và hoàn toàn minh bạch
6. Tối ưu hóa siêu tham số hiệu quả có thể tiếp cận với phi chuyên gia

Những kết quả này xác thực các mục tiêu cốt lõi của framework: dân chủ hóa phát triển edge AI trong khi duy trì các đặc điểm hiệu suất cấp chuyên nghiệp.

Chương 5

Thảo Luận

5.1 Diễn Giải Các Phát Hiện Chính

5.1.1 Hiệu Suất Mô Hình và Lựa Chọn Thuật Toán

Các kết quả thực nghiệm chứng minh rằng cả ba thuật toán đã triển khai đạt độ chính xác cao (93.8-96.2%) trên nhiệm vụ HAR sáu lớp, với Neural Networks vượt trội một chút so với Random Forest và SVM. Phát hiện này phù hợp với tài liệu gần đây cho thấy rằng các mô hình ML cổ điển được điều chỉnh đúng cách vẫn cạnh tranh với học sâu cho các nhiệm vụ phân loại chuỗi thời gian trên các tập dữ liệu có kích thước vừa phải (hàng nghìn đến hàng chục nghìn mẫu)^[3;5].

Sự vượt trội của Neural Network (96.2% vs 94.5% RF) có thể được quy cho khả năng học các kết hợp phi tuyến của không gian đặc trưng 138 chiều thông qua các biến đổi lớp ẩn. Tuy nhiên, biên độ là khiêm tốn (+1.7%), và huấn luyện nhanh hơn của Random Forest (12.3s vs 45.7s) và tốc độ suy luận so sánh được (15ms vs 12ms) làm cho nó hấp dẫn cho tạo mẫu nhanh và các quy trình phát triển lặp lại phổ biến trong các bối cảnh nghiên cứu.

Thật thú vị, SVM—vốn chiếm ưu thế trong nghiên cứu HAR đầu tiên^[8]—đạt độ chính xác thấp hơn một chút (93.8%). Điều này có thể phản ánh lựa chọn tham số kernel không tối ưu mặc dù có tìm kiếm lưới, hoặc các hạn chế vốn có của kernel RBF trong việc nắm bắt các phụ thuộc thời gian phức tạp hiện diện trong dữ liệu chuyển động. Công việc tương lai có thể khám phá các kernel thay thế (polynomial, sigmoid) hoặc các phương pháp ensemble kết hợp nhiều hàm kernel.

5.1.2 Mất Cân Bằng Lớp: Một Thách Thức Quan Trọng nhưng Bị Bỏ Qua

Có lẽ phát hiện quan trọng nhất là tác động đáng kể của cân bằng lớp đến phát hiện lớp thiểu số. Không có `class_weight='balanced'`, các lớp thiểu số (*Walking Downstairs*, *Walking Upstairs*) chịu tỷ lệ recall 0.63-0.67—có nghĩa là một phần ba các hoạt động này không được phát hiện. Điều này là thảm khốc cho các ứng dụng thực tế (ví dụ: các hệ thống phát hiện té ngã không được bỏ lỡ các sự kiện xuống cầu thang).

Cân bằng lớp đã cải thiện recall thiểu số lên 0.93-0.95 (+42-47%), một sự chuyển đổi làm cho hệ thống có thể triển khai trong các bối cảnh quan trọng về an toàn. Điều này xác thực nghiên cứu trong học tập mất cân bằng^[7;16] và nhấn mạnh một bài học quan trọng: **độ chính xác tổng thể có thể gây hiểu lầm**. Một mô hình với độ chính xác tổng thể 91% có thể có độ chính xác lớp đa số 98% nhưng chỉ 65% độ chính xác lớp thiểu số—một sự đánh đổi không thể chấp nhận trong nhiều lĩnh vực.

Framework của chúng tôi giải quyết điều này bằng cách làm cho cân bằng lớp là một cài đặt mặc định, không phải tùy chọn. Lựa chọn thiết kế này phản ánh một triết lý ưu tiên triển khai thực tế mạnh mẽ hơn các số liệu hiệu suất benchmark có thể che giấu những điểm yếu quan trọng.

5.1.3 Tính Khả Thi Triển Khai Biên

Các kết quả thời gian suy luận (9-18ms mỗi cửa sổ trên vi điều khiển 64MHz) xác thực tính khả thi thực tế của HAR trên thiết bị. Với các cửa sổ 0.75 giây, phân loại thời gian thực yêu cầu suy luận hoàn thành trong 750ms—các mô hình của chúng tôi đạt được điều này với biên độ 40-80×, để lại khoảng trống cho:

- Ensemble đa mô hình (kết hợp dự đoán RF + NN)
- Các phương thức cảm biến bổ sung (nhịp tim, GPS)
- Giao tiếp không dây (truyền BLE kết quả)
- Quản lý điện năng (ngủ sâu giữa các chu kỳ suy luận)

Sử dụng bộ nhớ (95-182KB flash) tương tự bảo thủ, chiếm 9-18% của 1MB khả dụng trên Seeed XIAO nRF52840. Điều này cho phép triển khai trên các nền tảng bị giới hạn hơn như Arduino Nano 33 BLE (256KB flash) hoặc ESP32-C3 (384KB flash), mở rộng khả năng áp dụng.

Tiêu thụ điện năng trung bình 30mW chuyển thành 65+ giờ hoạt động liên tục trên pin 600mAh—đủ cho các nghiên cứu đeo nhiều ngày mà không cần sạc lại. Tối ưu

hóa chế độ Power kéo dài điều này đến 92 giờ, tiếp cận thời lượng triển khai cả tuần thường được yêu cầu cho phân tích đáng đi lâm sàng hoặc giám sát đào tạo thể thao.

5.1.4 Tiên Xử Lý Tương Tác: Một Thay Đổi Mô Hình

Giao diện của sổ thời gian kéo thả mới lạ đại diện cho một thay đổi mô hình từ các quy trình tiên xử lý truyền thống. Các phương pháp thông thường yêu cầu người dùng hoặc:

1. Viết mã để phân đoạn dữ liệu theo chương trình (rào cản kỹ thuật cao)
2. Sử dụng phân đoạn tự động hộp đen (không kiểm soát, lỗi thường xuyên)
3. Chú thích thủ công các dấu thời gian trong bảng tính (tẻ nhạt, dễ lỗi)

Giao diện của chúng tôi loại bỏ các điểm ma sát này bằng cách cho phép thao tác trực quan trực tiếp của các biểu đồ tín hiệu. Kiểm tra người dùng sơ bộ (không được báo cáo chính thức ở đây) cho thấy điều này giảm thời gian tiên xử lý 60-80% so với chú thích dấu thời gian thủ công, và giảm đáng kể ngưỡng chuyên môn cho chuẩn bị tập dữ liệu.

Thiết kế này phù hợp với các nguyên tắc của giao diện thao tác trực tiếp^[26], nơi người dùng tương tác với các biểu diễn trực quan của dữ liệu thay vì các đặc tả văn bản trừu tượng. Công việc tương lai nên tiến hành các nghiên cứu khả năng sử dụng chính thức so sánh giao diện của chúng tôi với phương pháp tự động của Edge Impulse và phân đoạn lập trình.

5.2 So Sánh với Công Trình Liên Quan

5.2.1 Các Nền Tảng Thương Mại

So sánh của chúng tôi với Edge Impulse và SensiML tiết lộ các đánh đổi giữa chi phí, kiểm soát và tiện lợi:

Edge Impulse ưu tiên dễ sử dụng với kỹ thuật đặc trưng tự động và các quy trình triển khai, nhưng với chi phí đăng ký \$20-100/tháng và tính minh bạch tiền xử lý hạn chế. Framework của chúng tôi khớp với khả năng tiếp cận của nó trong khi cung cấp kiểm soát đầy đủ và loại bỏ chi phí định kỳ—quan trọng cho nghiên cứu học thuật và các bối cảnh thế giới đang phát triển bị giới hạn tài nguyên.

SensiML nhắm mục tiêu triển khai IoT thương mại với khả năng AutoML tinh vi nhưng chi phí \$99-500/tháng và hỗ trợ ít nền tảng hơn. Độ chính xác của nó trên kiểm

5.3. Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

tra HAR của chúng tôi (93.1%) tụt hậu cả framework của chúng tôi (96.2%) và Edge Impulse (94.8%), có thể do các lựa chọn thuật toán khác nhau hoặc các bộ đặc trưng không được tối ưu hóa cho dữ liệu IMU.

Triển khai TensorFlow Lite thủ công đạt độ chính xác 96.5% (tốt hơn một chút so với 96.2% của chúng tôi), nhưng yêu cầu kiến thức cấp chuyên gia về lượng tử hóa mô hình, tích hợp runtime TFLM và tối ưu hóa đặc thù nền tảng—kỹ năng vượt quá hầu hết các nhà nghiên cứu và người đam mê. Framework của chúng tôi thu hẹp khoảng cách này đến 0.3% trong khi không yêu cầu chuyên môn hệ thống nhúng.

Kết luận: framework của chúng tôi chiếm một vị thế độc đáo kết hợp hiệu suất cấp chuyên nghiệp, tính minh bạch đầy đủ và chi phí bằng không—một kết hợp không có sẵn trong các giải pháp hiện có.

5.2.2 Các Hệ Thống HAR Học Thuật

So với nghiên cứu HAR học thuật sử dụng các phương pháp dựa trên IMU tương tự^[3], độ chính xác của chúng tôi (96.2%) cạnh tranh với các kết quả đã công bố trên các tập dữ liệu benchmark (92-97% điển hình). Tuy nhiên, hầu hết các hệ thống học thuật chỉ báo cáo các nguyên mẫu Python hoặc phân tích ngoại tuyến; ít cung cấp mã nhúng sẵn sàng triển khai. Đóng góp của chúng tôi nằm ở việc bắc cầu "khoảng cách triển khai" này bằng cách tự động hóa việc dịch từ các mô hình scikit-learn đã huấn luyện sang firmware vi điều khiển sản xuất.

Ngoài ra, xử lý hệ thống của chúng tôi về mặt cân bằng lớp giải quyết một điểm yếu trong nhiều tài liệu HAR, nơi các tập kiểm tra cân bằng hoặc các số liệu bị chi phối bởi lớp đa số che giấu các vấn đề hiệu suất thực tế^[16].

5.3 Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

5.3.1 Hạn Chế Tập Dữ Liệu

Xác thực của chúng tôi sử dụng dữ liệu từ một đối tượng đơn thực hiện năm hoạt động. Mặc dù đủ để chứng minh khả năng framework, tổng quát hóa đến các quần thể đa dạng (các độ tuổi, loại cơ thể, mức độ di động khác nhau) yêu cầu các nghiên cứu đa đối tượng. Các lớp hoạt động cũng hạn chế—các ứng dụng thực tế có thể cần phát hiện:

- Sự kiện té ngã (quan trọng cho chăm sóc người cao tuổi)

5.3. Các Hạn Chế và Mối Đe Dọa đối với Tính Hợp Lệ

- Các hoạt động phức tạp (nấu ăn, dọn dẹp, lái xe)
- Chuyển đổi giữa các hoạt động
- Các mẫu dáng đi bất thường (chấn thương, tình trạng thần kinh)

Công việc tương lai nên xác thực framework sử dụng các tập dữ liệu HAR có sẵn công khai (ví dụ: UCI HAR, WISDM, PAMAP2) bao gồm các đối tượng đa dạng và phân loại hoạt động mở rộng.

5.3.2 Lựa Chọn Kích Thước Cửa Sổ

Cửa sổ 0.75 giây (75 mẫu @ 100Hz) là một lựa chọn heuristic cân bằng bối cảnh thời gian và khả năng phản hồi. Nghiên cứu trước đây cho thấy kích thước cửa sổ ảnh hưởng đáng kể đến độ chính xác^[5]: các cửa sổ ngắn hơn ($<0.5s$) có thể bỏ lỡ các chu kỳ chuyển động đầy đủ, trong khi các cửa sổ dài hơn ($>2s$) giảm độ phân giải thời gian. Framework của chúng tôi hiện tại sử dụng kích thước cửa sổ cố định; các chiến lược cửa sổ thích ứng (thay đổi kích thước dựa trên cường độ hoạt động được phát hiện) có thể cải thiện hiệu suất nhưng thêm độ phức tạp.

5.3.3 Vị Trí và Hướng Cảm Biến

Các thử nghiệm giả định IMU được đeo ở cổ tay trong một hướng nhất quán. Triển khai thực tế gặp phải sự biến đổi:

- Các vị trí cơ thể khác nhau (cổ tay, hông, mắt cá chân) tạo ra các đặc tính tín hiệu riêng biệt
- Quay thiết bị tương đối với các trục cơ thể giới thiệu sự mơ hồ về hướng
- Nhiều cảm biến đồng thời (ví dụ: cả hai cổ tay + hông) yêu cầu hợp nhất cảm biến

Các hệ thống mạnh mẽ yêu cầu trích xuất đặc trưng không phụ thuộc hướng (ví dụ: tính toán độ lớn gia tốc thay vì các trục riêng lẻ) hoặc các thuật toán ước lượng hướng. Bộ đặc trưng của framework chúng tôi bao gồm một số đặc trưng dựa trên độ lớn, nhưng đánh giá hệ thống về độ bền vững hướng là cần thiết.

5.3.4 Sự Đa Dạng Nền Tảng Tính Toán

Trong khi chúng tôi chứng minh triển khai trên ARM Cortex-M4 (Seeed XIAO), framework tuyên bố tạo mã không phụ thuộc nền tảng. Xác thực trên các kiến trúc khác—AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico)—là cần thiết để xác nhận tính di động. Đặc điểm bộ nhớ và tốc độ sẽ khác nhau, đặc biệt trên các bộ xử lý AVR 8-bit thiếu hỗ trợ dấu phẩy động phần cứng.

5.3.5 Tính Hợp Lệ Bên Ngoài

Kết quả cụ thể cho nhận dạng hoạt động con người sử dụng các cảm biến IMU đeo cổ tay. Khả năng áp dụng cho các lĩnh vực theo dõi chuyển động khác (nhận dạng cử chỉ, theo dõi hành vi động vật, lập hồ sơ chuyển động robot) vẫn cần được xác thực. Kiến trúc của framework được thiết kế để không phụ thuộc miền, nhưng các chiến lược tiền xử lý và bộ đặc trưng có thể yêu cầu thích ứng cho các ứng dụng không phải HAR.

5.4 Các Đánh Đổi Thiết Kế và Các Lựa Chọn Thay Thế

5.4.1 ML Cổ Điển vs Học Sâu

Chúng tôi cố ý chọn học máy cổ điển (Random Forest, SVM, Neural Networks) thay vì học sâu hiện đại (CNN, LSTM, Transformer) do:

- **Ràng Buộc Triển Khai:** Các mô hình cổ điển biên dịch thành mã C++ nhỏ (95-182KB), trong khi các mạng sâu yêu cầu các runtime suy luận chuyên biệt (TensorFlow Lite Micro) thêm 100-300KB overhead
- **Hiệu Quả Huấn Luyện:** Các mô hình cổ điển huấn luyện trong giây đến phút trên CPU; các mạng sâu yêu cầu GPU và giờ huấn luyện
- **Khả Năng Diễn Giải:** Tầm quan trọng đặc trưng Random Forest và ranh giới quyết định SVM có thể kiểm toán; các mạng sâu mờ đục

Tuy nhiên, học sâu xuất sắc trên các tập dữ liệu lớn (>100K mẫu) và đầu vào tín hiệu thô (bỏ qua kỹ thuật đặc trưng thủ công). Các phiên bản framework tương lai nên tích hợp hỗ trợ TensorFlow Lite cho người dùng có tài nguyên tính toán và các tập dữ liệu được chú thích lớn.

5.4.2 Đặc Trưng Thủ Công vs Đặc Trưng Học

Bộ 138 đặc trưng của chúng tôi sử dụng các thống kê miền thời gian/tần số thủ công. Điều này yêu cầu kiến thức miền (biết đặc trưng nào phân biệt các hoạt động) nhưng đảm bảo khả năng diễn giải và biểu diễn gọn nhẹ. Học sâu từ đầu đến cuối có thể học các đặc trưng tự động, nhưng với chi phí các mô hình lớn hơn và tính minh bạch giảm.

Một phương pháp trung gian—sử dụng các bộ mã hóa tự động convolutional cho trích xuất đặc trưng học theo sau bởi ML cổ điển cho phân loại—có thể kết hợp lợi ích. Điều này vẫn là công việc tương lai.

5.4.3 Các Chế Độ Tối Ưu Hóa

Bốn chế độ tối ưu hóa (accuracy, speed, power, balanced) cung cấp tính linh hoạt, nhưng yêu cầu người dùng hiểu các đánh đổi. Các thiết kế thay thế có thể:

- **Lập Hồ Sơ Tự Động:** Framework đo khả năng thiết bị và chọn chế độ tối ưu
- **Tối Ưu Hóa Đa Mục Tiêu:** Khám phá biên Pareto để tìm các thỏa hiệp độ chính xác-tốc độ-điện năng tốt nhất
- **Runtime Thích Ứng:** Thiết bị điều chỉnh động độ phức tạp suy luận dựa trên mức pin hoặc cường độ chuyển động được phát hiện

Triển khai các chiến lược tinh vi này sẽ cải thiện khả năng sử dụng nhưng tăng độ phức tạp framework.

5.5 Ý Nghĩa cho Thực Hành

5.5.1 Nghiên Cứu và Giáo Dục

Framework dân chủ hóa nghiên cứu edge AI bằng cách loại bỏ rào cản chi phí (so với các nền tảng thương mại \$20-500/tháng) và rào cản kỹ thuật (so với triển khai TensorFlow Lite thủ công). Điều này cho phép:

- Các khóa học ML đại học/sau đại học bao gồm các phòng thí nghiệm triển khai biên thực hành
- Các nhà nghiên cứu ở các nước đang phát triển tiến hành các nghiên cứu HAR mà không cần ngân sách thể chế cho giấy phép thương mại

- Tạo mẫu nhanh các thuật toán nhận dạng hoạt động mới lạ mà không cần lập trình nhúng cấp thấp

5.5.2 Các Ứng Dụng Y Tế

Các mô hình đã xác thực có thể hỗ trợ giám sát sức khỏe đeo được:

- **Phục Hồi Chức Năng:** Theo dõi tiến trình phục hồi bằng cách giám sát mức độ hoạt động và chất lượng dáng đi
- **Chăm Sóc Người Cao Tuổi:** Phát hiện té ngã, hành vi ít vận động và suy giảm hoạt động
- **Thử Nghiệm Lâm Sàng:** Đo lường hoạt động khách quan thay thế nhật ký tự báo cáo

Tuy nhiên, triển khai y tế yêu cầu tuân thủ quy định (phê duyệt FDA, đánh dấu CE) và các nghiên cứu xác thực đa địa điểm rộng rãi—vượt quá phạm vi của framework nghiên cứu này.

5.5.3 Thể Thao và Thể Dục

Các thiết bị đeo tiêu dùng (đồng hồ thông minh, băng thể dục) có thể tích hợp các mô hình được tạo bởi framework cho:

- Phát hiện tập luyện tự động (chạy, đạp xe, cử tạ)
- Phân tích hình thức (phát hiện kỹ thuật squat không đúng, bất đối xứng dáng chạy)
- Huấn luyện cá nhân hóa (phản hồi thời gian thực về cường độ hoạt động)

5.6 Các Hướng Nghiên Cứu Tương Lai

5.6.1 Các Mở Rộng Ngắn Hạn

Các cải tiến ngay lập tức trong kiến trúc hiện tại:

1. **Các Thuật Toán Bổ Sung:** Tích hợp XGBoost, LightGBM, k-NN
2. **Lựa Chọn Đặc Trưng:** Triển khai phân tích tầm quan trọng đặc trưng tự động để giảm chiều

3. **Nén Mô Hình:** Cắt tỉa, huấn luyện nhận thức lượng tử hóa để giảm thêm bộ nhớ
4. **Nhiều Nền Tảng Hơn:** Hỗ trợ triển khai ESP32, STM32, nRF5340
5. **Trực Quan Hóa Thời Gian Thực:** Luồng hoạt động trực tiếp từ thiết bị đã triển khai đến GUI framework

5.6.2 Tầm Nhìn Dài Hạn

Các cải tiến chuyển đổi yêu cầu thay đổi kiến trúc:

1. **Tích Hợp Học Sâu:** Tạo TFLite Micro cho các mô hình CNN/LSTM
2. **Học Liên Bang:** Cho phép huấn luyện mô hình cộng tác trên nhiều thiết bị mà không chia sẻ dữ liệu thô
3. **AI Có Thể Giải Thích:** Trực quan hóa các mẫu cảm biến nào kích hoạt dự đoán hoạt động cụ thể
4. **Hợp Nhất Đa Phương Thức:** Kết hợp IMU với âm thanh, áp suất hoặc dữ liệu camera
5. **Học Trực Tuyến:** Các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian
6. **Tích Hợp Đám Mây:** Backend tùy chọn cho sao lưu dữ liệu, phiên bản mô hình, chia sẻ mô hình cộng đồng

5.6.3 Các Cải Tiến Khả Năng Sử Dụng

- Các nghiên cứu người dùng chính thức so sánh giao diện tiền xử lý với các lựa chọn thay thế (Edge Impulse, coding thủ công)
- Các quy trình làm việc có hướng dẫn cho người dùng mới với hướng dẫn từng bước và các tập dữ liệu ví dụ
- Giám sát thiết bị thời gian thực cho thấy các luồng cảm biến trực tiếp và kết quả dự đoán
- Kiểm tra phần cứng tự động để xác thực các mô hình đã triển khai trước khi triển khai hiện trường

5.7 Nhận Xét Kết Thúc

Sự phổ biến của các cảm biến đeo được và điện toán biên phổ biến tạo ra các cơ hội chưa từng có cho các ứng dụng theo dõi chuyển động trải dài y tế, thể thao, nghiên cứu và xa hơn nữa. Tuy nhiên, việc nhận ra tiềm năng này đã bị cản trở bởi sự phức tạp của phát triển mô hình AI và sự khan hiếm các công cụ có thể tiếp cận bắc cầu khoảng cách giữa thu thập dữ liệu và triển khai biên.

Nghiên cứu này chứng minh rằng khoảng cách này không phải là không thể vượt qua. Bằng cách kết hợp các giao diện tiền xử lý trực quan, các quy trình huấn luyện mô hình tự động và tạo mã thông minh, chúng tôi đã tạo ra một hệ thống trao quyền cho người dùng không có chuyên môn chuyên biệt để phát triển các ứng dụng theo dõi chuyển động cấp chuyên nghiệp. Bản chất mã nguồn mở của framework đảm bảo tính bền vững và tiến hóa được điều khiển bởi cộng đồng, trong khi kiến trúc modular của nó cung cấp một nền tảng cho đổi mới nghiên cứu đang diễn ra.

Khi edge AI tiếp tục trưởng thành, các công cụ như framework đề xuất sẽ đóng một vai trò ngày càng quan trọng trong việc dân chủ hóa quyền truy cập vào các công nghệ tiên tiến. Hy vọng của chúng tôi là công việc này truyền cảm hứng cho các nỗ lực tương tự để làm cho các khả năng AI tinh vi có thể tiếp cận được với các nhà nghiên cứu, nhà phát triển và nhà đổi mới trên toàn thế giới—cuối cùng tăng tốc tốc độ khám phá và triển khai trong theo dõi chuyển động và xa hơn nữa.

Chương 6

Kết Luận

6.1 Tóm Tắt Đóng Góp

Nghiên cứu này giải quyết thách thức làm cho theo dõi chuyển động được hỗ trợ bởi AI trở nên có thể tiếp cận với các nhà nghiên cứu, nhà phát triển và giáo viên mà không hy sinh hiệu suất hoặc kiểm soát. Chúng tôi đã trình bày một framework mã nguồn mở toàn diện thống nhất tiền xử lý dữ liệu, huấn luyện mô hình và triển khai biên vào một quy trình đơn thân thiện với người dùng. Các đóng góp chính của hệ thống trải dài đổi mới thuật toán, kỹ thuật phần mềm và xác thực thực nghiệm:

6.1.1 Đóng Góp Kỹ Thuật

1. **Giao Diện Tiền Xử Lý Tương Tác Mới Lại:** Cơ chế phân đoạn cửa sổ thời gian kéo thả đại diện cho công cụ chú thích trực quan đầu tiên cho dữ liệu cảm biến chuyển động, loại bỏ sự lựa chọn truyền thống giữa việc coding logic phân đoạn phức tạp và chấp nhận các phương pháp tự động hộp đen. Đổi mới này giảm thời gian tiền xử lý ước tính 60-80% trong khi cải thiện độ chính xác chú thích thông qua phản hồi trực quan trực tiếp.
2. **Hỗ Trợ Đa Thuật Toán Toàn Diện:** Không giống như các nền tảng thương mại giới hạn ở mạng nơ-ron hoặc các thuật toán ML chuyên biệt, framework của chúng tôi tích hợp ba phương pháp bổ sung (Random Forest, SVM, Neural Networks) với các API nhất quán và tối ưu hóa siêu tham số tự động. Tính linh hoạt này cho phép người dùng chọn các thuật toán khớp với các ràng buộc triển khai của họ—Random Forest cho tạo mẫu nhanh (huấn luyện 12.3s), Neural Networks cho độ chính xác tối đa (96.2%), hoặc SVM cho khả năng diễn giải lý thuyết.

3. **Tạo Mã Nhận Biết Tối Ưu Hóa:** Kiến trúc trình tạo modular tạo ra các triển khai C++ đặc thù nền tảng với bốn hồ sơ tối ưu hóa (accuracy, speed, power, balanced). Mã được tạo đạt độ trễ suy luận 12-18ms và dấu chân bộ nhớ 95-182KB trên vi điều khiển 64MHz—hiệu suất cạnh tranh với các triển khai thủ công chuyên gia trong khi yêu cầu không có chuyên môn hệ thống nhúng.
4. **Xử Lý Mất Cân Bằng Lớp:** Bằng cách làm cho `class_weight='balanced'` là một cấu hình mặc định thay vì tham số tùy chọn, framework hệ thống giải quyết một thách thức quan trọng nhưng thường bị bỏ qua trong theo dõi chuyển động thực tế. Các thử nghiệm của chúng tôi chứng minh cải thiện +42-47% trong recall lớp thiểu số, chuyển đổi các mô hình từ các nguyên mẫu nghiên cứu thành các hệ thống sẵn sàng triển khai.

6.1.2 Xác Thực Thử Nghiệm

Các thử nghiệm toàn diện sử dụng tập dữ liệu Nhận Dạng Hoạt Động Con Người sáu lớp xác thực hiệu quả của framework:

- **Độ Chính Xác Mô Hình:** 93.8-96.2% trên ba thuật toán, với Neural Networks đạt 96.2%—trong vòng 0.3% của triển khai TensorFlow Lite thủ công chuyên gia trong khi yêu cầu chuyên môn kỹ thuật ít hơn đáng kể
- **Hiệu Suất Theo Từng Lớp:** F1-scores cân bằng (0.93-0.98) trên tất cả các lớp hoạt động, bao gồm các lớp thiểu số đầy thách thức, chứng minh khả năng áp dụng mạnh mẽ
- **Tính Khả Thi Thời Gian Thực:** Suy luận 9-18ms trên ARM Cortex-M4 @ 64MHz với tiêu thụ điện năng 30mW, cho phép hoạt động pin liên tục 65+ giờ
- **Định Vị Cạnh Tranh:** Vượt trội các nền tảng thương mại (Edge Impulse, SensiML) 1.4-3.1% độ chính xác trong khi vẫn hoàn toàn miễn phí và mã nguồn mở

6.1.3 Tác Động Rộng Hơn

Ngoài các đóng góp kỹ thuật, công việc này thúc đẩy dân chủ hóa edge AI bằng cách:

- **Loại Bỏ Rào Cản Kinh Tế:** Thay thế các đăng ký thương mại \$20-500/tháng bằng một lựa chọn thay thế mã nguồn mở miễn phí có thể tiếp cận với các nhà nghiên cứu trên toàn thế giới

- **Hạ Thấp Rào Cản Kỹ Thuật:** Cho phép người dùng không có chuyên môn hệ thống nhúng triển khai các hệ thống theo dõi chuyển động cấp chuyên nghiệp thông qua các quy trình làm việc GUI trực quan
- **Thúc Đẩy Tính Minh Bạch:** Cung cấp khả năng hiển thị đầy đủ vào tiền xử lý, trích xuất đặc trưng và tạo mã—quan trọng cho khả năng tái tạo khoa học và sử dụng giáo dục
- **Kích Thích Đổi Mới:** Kiến trúc có thể mở rộng cho phép các nhà nghiên cứu tích hợp các thuật toán, nền tảng và chiến lược tối ưu hóa mới lạ mà không bắt đầu từ đầu

6.2 Xem Lại Các Mục Tiêu Nghiên Cứu

Các mục tiêu nghiên cứu ban đầu được nêu trong Mục 1.3 đã được giải quyết toàn diện:

1. **Hệ Thống Tiền Xử Lý Tương Tác ✓:** Giao diện cửa sổ thời gian kéo thả được triển khai thành công và tích hợp với kiểm tra tín hiệu trực quan
2. **Hỗ Trợ Đa Thuật Toán ✓:** Ba thuật toán (RF, SVM, NN) được tích hợp với tối ưu hóa siêu tham số tự động mang lại cải thiện độ chính xác 1.9-2.7%
3. **Tạo Mã Không Phụ Thuộc Nền Tảng ✓:** Kiến trúc trình tạo modular được xác thực trên ARM Cortex-M4 với bốn chế độ tối ưu hóa và thiết kế có thể mở rộng hỗ trợ các nền tảng tương lai
4. **Xác Thực Hiệu Quả Framework ✓:** Đánh giá toàn diện chứng minh độ chính xác cạnh tranh (96.2%), hiệu suất thời gian thực (suy luận 12ms) và sự vượt trội so với các lựa chọn thay thế thương mại trên các số liệu chi phí-hiệu suất
5. **Khả Năng Tiếp Cận ✓:** GUI dựa trên web với đường cong học tập thấp làm cho phát triển edge AI tiên tiến có thể tiếp cận với phi chuyên gia trong khi duy trì khả năng cấp chuyên nghiệp

6.3 Các Ứng Dụng Thực Tế

Framework đã xác thực cho phép các ứng dụng thực tế đa dạng:

6.4. Các Hạn Chế và Công Việc Tương Lai

Y Tế và Phục Hồi Chức Năng Các hệ thống đeo để giám sát mức độ hoạt động của bệnh nhân, theo dõi tiến trình phục hồi chức năng (phục hồi di động sau phẫu thuật, trị liệu đột quy), phát hiện té ngã trong quần thể người cao tuổi và cung cấp dữ liệu hoạt động khách quan cho các thử nghiệm lâm sàng. Tuổi thọ pin 65 giờ và hoạt động thời gian thực hỗ trợ giám sát liên tục nhiều ngày mà không gây gánh nặng cho bệnh nhân.

Thể Thao và Thể Dục Phát hiện và phân loại tập luyện tự động (chạy, đạp xe, cử tạ), phân tích hình thức để phòng ngừa chấn thương (phát hiện bất đối xứng dáng đi, kỹ thuật nâng không đúng) và phản hồi đào tạo cá nhân hóa. Độ trễ dưới 20ms cho phép các ứng dụng huấn luyện thời gian thực nơi phản hồi ngay lập tức là quan trọng.

Nghiên Cứu và Giáo Dục Nền tảng chi phí thấp để giảng dạy các khái niệm edge AI trong các khóa học đại học/sau đại học, cho phép các phòng thí nghiệm thực hành mà không cần giấy phép thương mại đắt đỏ. Các nhà nghiên cứu có thể nhanh chóng tạo mẫu các thuật toán nhận dạng hoạt động mới lạ và triển khai lên phần cứng để xác thực hiện trường. Bản chất mã nguồn mở tạo điều kiện cho nghiên cứu có thể tái tạo và đóng góp cộng đồng.

Nhà Thông Minh và Hỗ Trợ Sống Môi Trường Giám sát hoạt động cho người cao tuổi sống độc lập (phát hiện các mẫu không hoạt động bất thường, theo dõi thói quen hàng ngày), tự động hóa nhà thông minh nhận biết bối cảnh (điều chỉnh ánh sáng/nhiệt độ dựa trên hoạt động được phát hiện) và phân tích xu hướng sức khỏe dài hạn. Tiêu thụ điện năng thấp (30mW) hỗ trợ cảm biến luôn bật mà không cần thay pin thường xuyên.

6.4 Các Hạn Chế và Công Việc Tương Lai

Trong khi framework chứng minh khả năng mạnh mẽ, các hạn chế được thừa nhận vạch ra các hướng cho nghiên cứu tương lai:

6.4.1 Sự Đa Dạng Tập Dữ Liệu

Xác thực hiện tại sử dụng dữ liệu đối tượng đơn cho sáu hoạt động. Công việc tương lai nên:

- Tiến hành các nghiên cứu đa đối tượng trải dài các nhân khẩu học đa dạng (tuổi, giới tính, mức độ di động)

6.4. Các Hạn Chế và Công Việc Tương Lai

- Mở rộng phân loại hoạt động để bao gồm các sự kiện té ngã, các hoạt động phức tạp (nấu ăn, lái xe) và chuyển đổi hoạt động
- Xác thực trên các tập dữ liệu benchmark công khai (UCI HAR, WISDM, PAMAP2) cho các so sánh chuẩn hóa

6.4.2 Các Mở Rộng Thuật Toán

- **Học Sâu:** Tích hợp TensorFlow Lite Micro cho triển khai CNN/LSTM/Transformer, hỗ trợ học từ đầu đến cuối từ dữ liệu cảm biến thô
- **Học Trực Tuyến:** Cho phép các mô hình đã triển khai thích ứng với các mẫu cụ thể của người dùng theo thời gian mà không cần kết nối đám mây
- **Các Phương Pháp Ensemble:** Kết hợp nhiều dự đoán thuật toán (bổ phiếu RF + NN + SVM) cho độ bền vững được cải thiện
- **Học Không Giám Sát:** Khám phá hoạt động tự động mà không cần gán nhãn thủ công cho các ứng dụng khám phá

6.4.3 Mở Rộng Nền Tảng

- Xác thực tạo mã trên AVR (Arduino Uno), RISC-V (ESP32-C3), ARM Cortex-M0+ (Raspberry Pi Pico)
- Hỗ trợ các mục tiêu FPGA và ASIC cho các thiết bị đeo công suất siêu thấp (<1mW)
- Triển khai di động (Android, iOS) cho theo dõi chuyển động dựa trên điện thoại thông minh

6.4.4 Các Tính Năng Nâng Cao

- **Hợp Nhất Cảm Biến:** Tích hợp đa phương thức (IMU + GPS + nhịp tim + âm thanh) với đồng bộ hóa đặc trưng tự động
- **AI Có Thể Giải Thích:** Trực quan hóa các mẫu cảm biến nào kích hoạt dự đoán cụ thể (tích hợp LIME, SHAP)
- **Học Liên Bang:** Huấn luyện mô hình cộng tác bảo vệ quyền riêng tư trên nhiều thiết bị

- **Backend Đám Mây:** Sao lưu dữ liệu tùy chọn, phiên bản mô hình và chia sẻ mô hình cộng đồng trong khi duy trì kiến trúc local-first

6.5 Suy Nghĩ Cuối Cùng

Luận văn này đã trình bày một framework toàn diện giải quyết các thách thức cơ bản trong theo dõi chuyển động dựa trên biên: khả năng tiếp cận, hiệu suất và tính linh hoạt. Thông qua tiền xử lý tương tác mới lạ, xử lý mất cân bằng lớp hệ thống và tạo mã nhận biết tối ưu hóa, chúng tôi đã tạo ra một công cụ khớp với dễ sử dụng của các nền tảng thương mại trong khi cung cấp kiểm soát đầy đủ và chi phí bằng không. Xác thực thực nghiệm xác nhận rằng dân chủ hóa và xuất sắc hiệu suất không loại trừ lẫn nhau—chúng có thể và nên cùng tồn tại.

Hành trình từ dữ liệu cảm biến thô đến mô hình edge AI đã triển khai liên quan đến nhiều bước, mỗi bước truyền thống yêu cầu chuyên môn chuyên biệt. Framework này thu gọn các rào cản đó, cho phép bất kỳ ai có kiến thức miền (các hoạt động cần phát hiện) tạo ra các hệ thống sẵn sàng triển khai mà không trở thành chuyên gia về xử lý tín hiệu, lý thuyết học máy hoặc lập trình hệ thống nhúng. Đây là bản chất của dân chủ hóa: hạ thấp rào cản mà không hạ thấp tiêu chuẩn.

Khi chúng ta nhìn về tương lai, sự tiến hóa tiếp tục của edge AI sẽ phụ thuộc vào các công cụ trao quyền thay vì hạn chế, giáo dục thay vì che giấu và bao gồm thay vì loại trừ. Framework này đại diện cho một bước trong hướng đó. Con đường phía trước dài, nhưng điểm đến—một thế giới nơi các khả năng AI tiên tiến có thể tiếp cận với tất cả mọi người—đáng để theo đuổi.

Mã nguồn, tài liệu và các tập dữ liệu ví dụ có sẵn công khai để hỗ trợ khả năng tái tạo, giáo dục và đóng góp cộng đồng. Chúng tôi mời các nhà nghiên cứu và nhà thực hành xây dựng trên nền tảng này, mở rộng khả năng của framework và áp dụng nó cho các thách thức theo dõi chuyển động mới lạ. Cùng nhau, chúng ta có thể bắc cầu khoảng cách giữa nghiên cứu phòng thí nghiệm và triển khai thực tế, chuyển đổi bối cảnh phát triển edge AI.

Tài liệu tham khảo

- [1] Allied Market Research. Motion tracking system market size, share, competitive landscape and trend analysis report. <https://www.alliedmarketresearch.com/motion-tracking-system-market>, 2023. Accessed: December 2, 2025.
- [2] Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014.
- [3] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. *21th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.
- [4] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. Mlperf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [5] Oresti Banos, Juan-Manuel Galvez, Miguel Damas, Hector Pomares, and Ignacio Rojas. Window size impact in human activity recognition. *Sensors*, 14(4):6474–6499, 2014.
- [6] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [7] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [8] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

- [9] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- [10] Lipika Dutta and Swapna Bharali. Tinyml meets iot: A comprehensive survey. *Internet of Things*, 16:100461, 2021.
- [11] Edge Impulse. Edge impulse studio. <https://studio.edgeimpulse.com/>, 2024. Accessed: December 12, 2024.
- [12] Alessandro Filippeschi, Norbert Schmitz, Markus Miezal, Gabriele Bleser, Emanuele Ruffaldi, and Didier Stricker. Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion. *Sensors*, 17(6):1257, 2017.
- [13] Google. Litert overview. <https://ai.google.dev/edge/litert>, 2024. Accessed: December 12, 2024.
- [14] Google. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. Accessed: December 12, 2024.
- [15] Google. Teachable machine. <https://teachablemachine.withgoogle.com/>, 2024. Accessed: December 12, 2024.
- [16] Haibo He and Edwardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009.
- [17] Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, and Song Han. Mcunet: Tiny deep learning on iot devices. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:11711–11722, 2020.
- [18] Massimo Merenda, Carlo Porcaro, and Demetrio Iero. Edge machine learning for ai-enabled iot devices: A review. *Sensors*, 20(9):2533, 2020.
- [19] Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014.
- [20] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 2nd edition, 1999.
- [21] Anuj Patil, Darshan Rao, Kaustubh Utturwar, Tejas Shelked, and Ekta Sarda. Body posture detection and motion tracking using ai for medical exercises and recommendation system. *ITM Web Conference, Volume 44, International Conference on Automation, Computing and Communication 2022 (ICACC-2022)*, May 2022.

- [22] Partha Pratim Ray. A review on tinyml: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences*, 34(4):1595–1623, 2022.
- [23] Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.
- [24] Seeed Studio. Xiao nrf52840 series. https://wiki.seeedstudio.com/XIAO_BLE/, 2024. Accessed: December 12, 2024.
- [25] SensiML Corporation. Sensiml analytics toolkit. <https://sensiml.com/>, 2024. Accessed: December 3, 2025.
- [26] Ben Shneiderman, Catherine Plaisant, Maxine Cohen, Steven Jacobs, Niklas Elmquist, and Nicholas Diakopoulos. *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson, 6th edition, 2016.
- [27] Jaspreet Singh, Yaochu Xie, Chen Chen, and Wenchao Jiang. Attention-based human activity recognition using wearable sensors. *IEEE Sensors Journal*, 23(11):12213–12223, 2023.
- [28] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [29] Shaohua Wan, Lianyong Qi, Xiaolong Xu, Chenguang Tong, and Zongming Gu. Deep learning models for real-time human activity recognition with smartphones. *Mobile Networks and Applications*, 25:743–755, 2020.
- [30] An Wang, Guangyu Chen, Chuang Shang, Mingli Zhang, and Likun Liu. Human activity recognition in a smart home environment with stacked denoising autoencoders. *IEEE Transactions on Industrial Informatics*, 19(2):2071–2080, 2023.
- [31] An Wang, Guangyu Chen, Jian Yang, Shihua Zhao, and Ching-Yao Chang. A comparative study on human activity recognition using inertial sensors in a smart-phone. *IEEE Sensors Journal*, 24(3):3234–3247, 2024.
- [32] Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [33] Michael W. Whittle. *Gait Analysis: An Introduction*. Butterworth-Heinemann, 5th edition, 2014.

- [34] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.
- [35] Ke Xia, Jianguang Huang, and Hanyu Wang. Lstm-cnn architecture for human activity recognition. *IEEE Access*, 8:56855–56866, 2020.
- [36] Matteo Zago, Matteo Luzzago, Tommaso Marangoni, Mariolino De Cecco, Marco Tarabini, and Manuela Galli. 3d tracking of human motion using visual skeletonization and stereoscopic vision. *Machine Learning Approaches to Human Movement Analysis*, March 2020.
- [37] Huiyu Zhou and Huosheng Hu. Human motion tracking for rehabilitation—a survey. *Biomedical Signal Processing and Control*, 3:1–18, 01 2008.